



RAPPORT DE STAGE

En vue de l'obtention du diplôme de Licence en Business Computing

Parcours Business Intelligence

Réalisé par : **Oussema Korbosli**

**« Conception et développement d'un système intelligent de gestion des rôles
et de pointage des employés avec tableau de bord décisionnel »**

Du 16 février au 16 juin 2026

Maître de stage : Melek Aziz Elmoula

Encadrant académique : Heni Abidi

Année universitaire 2025/2026

Résumé

Réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne**, ce projet de fin d'études porte sur la conception et le développement d'un **système intelligent de gestion des rôles et de pointage des employés**, doté d'un tableau de bord décisionnel (Business Intelligence). L'application centralise la gestion des agences, des employés, des clients et des objectifs commerciaux, gère les rôles et la sécurité des accès, assure le **pointage (présence) des employés**, et formalise les opérations sensibles au moyen de circuits de validation. Développée avec la plateforme **Jakarta EE 10** (Servlets, JSP, EJB, JPA/Hibernate) au-dessus d'une base **Oracle**, elle est complétée par un volet décisionnel : une chaîne **ETL** (Python) consolide les données dans un datamart qui alimente des vues exposées à **Microsoft Power BI**, un tableau de bord embarqué (effectifs, présence, production) et une **segmentation des agences** par apprentissage non supervisé (clustering). La solution améliore la centralisation des données, la traçabilité des opérations et l'aide à la décision.

Mots-clés : gestion des rôles, pointage des employés, Business Intelligence, ETL, aide à la décision, apprentissage automatique, clustering, Jakarta EE, Oracle, Power BI.

Abstract

Carried out within **BTK – Banque Tuniso-Koweïtienne**, this end-of-studies project deals with the design and development of an **intelligent system for managing employee roles and time-tracking (attendance)**, featuring a business-intelligence dashboard. The application centralizes the management of branches, employees, customers and commercial objectives, handles roles and access security, records **employee attendance (clock-in/clock-out)**, and formalizes sensitive operations through approval workflows. Built on the **Jakarta EE 10** platform (Servlets, JSP, EJB, JPA/Hibernate) over an **Oracle** database, it is complemented by a decision-support layer : a Python **ETL** pipeline consolidates data into a datamart that feeds analytical views exposed to **Microsoft Power BI**, an embedded dashboard (headcount, attendance, production) and a **branch segmentation** based on unsupervised machine learning (clustering). The solution im-

proves data centralization, operation traceability and decision-making.

Key words : role management, employee time-tracking, Business Intelligence, ETL, decision support, machine learning, clustering, Jakarta EE, Oracle, Power BI.

Dédicaces

Je dédie ce modeste travail :

*À mes chers **parents**,
pour leur amour, leurs sacrifices et leur soutien sans faille tout au long de mon parcours.
Aucun mot ne saurait exprimer ma gratitude.*

*À ma **famille**,
pour ses encouragements constants et sa présence dans les moments importants.*

*À mes **amis** et à mes **camarades**,
pour les bons moments partagés et leur soutien tout au long de ces années d'études.*

*À tous mes **enseignants**,
pour la qualité de la formation et le savoir qu'ils ont su me transmettre.*

À tous ceux qui m'ont aidé, de près ou de loin, à réaliser ce travail.

Oussema Korbosli

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie chaleureusement mon encadrant académique, **Heni Abidi**, pour ses conseils avisés, sa disponibilité et son suivi rigoureux tout au long de ce projet.

Mes remerciements s'adressent également à mon maître de stage, **Melek Aziz Elmoula**, Data Scientist au sein de la **BTK – Banque Tuniso-Koweïtienne**, pour la confiance accordée, l'accompagnement et le partage de l'expérience métier.

Je tiens aussi à remercier l'ensemble du corps enseignant et administratif de l'**Esprit School of Business** pour la qualité de la formation dispensée durant mon cursus en Licence Business Computing, parcours Business Intelligence.

Enfin, j'exprime ma reconnaissance à ma famille et à mes amis pour leur soutien constant et leurs encouragements.

Table des matières

Dédicaces	i
Remerciements	ii
Liste des acronymes	xii
Introduction générale	1
1 Cadre général du projet	2
1.1 Cadre du projet	2
1.2 Présentation de l'organisme d'accueil	2
1.2.1 Missions	3
1.2.2 Objectifs	4
1.3 Étude de l'existant	4
1.3.1 Description de la situation existante	4
1.3.2 Critique de la situation existante	4
1.3.3 Problématique	4
1.3.4 Solution proposée	5
1.3.5 Justification du choix de la solution	5
1.4 Méthodologie adoptée	5
1.4.1 Méthodes traditionnelles et leurs limites	5
1.4.2 Origines et apport des méthodes agiles	6
1.4.3 Comparaison des cadres agiles : Kanban et Scrum	6
1.4.4 Choix de la méthodologie : Scrum	6

1.4.5	Présentation de Scrum	7
2	Sprint 0 : Analyse et Spécification des Besoins	8
2.1	Étude du contexte	8
2.1.1	Identification des acteurs	8
2.1.2	Spécification des besoins	8
2.1.2.1	Besoins fonctionnels	8
2.1.2.2	Besoins non fonctionnels	9
2.2	Pilotage du projet avec Scrum	10
2.3	Backlog du produit	10
2.4	Diagramme de cas d'utilisation global	10
2.5	Diagramme de classe	14
2.6	Planification des sprints	14
2.7	Environnement de développement	14
2.7.1	Environnement matériel	14
2.7.2	Outil de rédaction du rapport	16
2.7.3	Environnement de développement logiciel	16
2.8	Architecture du système	19
2.8.1	Architecture globale	19
2.8.2	Architecture physique	19
2.8.3	Architecture logique	19
3	Sprint 1 : Authentification et gestion des rôles	23
3.1	Objectifs du Sprint	23
3.2	Spécification des besoins	23
3.2.1	Diagramme de cas d'utilisation du Sprint 1	23
3.2.2	Sprint Backlog du sprint 1	23

3.3	Analyse des besoins	25
3.3.1	Description textuelle des cas d'utilisation	25
3.4	Conception	26
3.4.1	Diagramme de séquence : « S'authentifier »	26
3.5	Réalisation : interfaces utilisateur	26
4	Sprint 2 : Gestion des agences et des employés	28
4.1	Objectifs du Sprint	28
4.2	Spécification des besoins	28
4.2.1	Diagramme de cas d'utilisation du Sprint 2	28
4.2.2	Sprint Backlog du sprint 2	28
4.3	Analyse des besoins	30
4.3.1	Description textuelle des cas d'utilisation	30
4.4	Conception	30
4.4.1	Diagramme de séquence : « Créer une agence »	30
4.5	Réalisation : interfaces utilisateur	31
5	Sprint 3 : Gestion des clients et des objectifs	33
5.1	Objectifs du Sprint	33
5.2	Spécification des besoins	33
5.2.1	Diagramme de cas d'utilisation du Sprint 3	33
5.2.2	Sprint Backlog du sprint 3	33
5.3	Analyse des besoins	35
5.3.1	Description textuelle des cas d'utilisation	35
5.4	Conception	35
5.4.1	Diagramme de séquence : « Gérer un client »	35
5.5	Réalisation : interfaces utilisateur	36

6	Sprint 4 : Pointage, demandes et notifications	38
6.1	Objectifs du Sprint	38
6.2	Spécification des besoins	38
6.2.1	Diagramme de cas d'utilisation du Sprint 4	38
6.2.2	Sprint Backlog du sprint 4	38
6.3	Analyse des besoins	40
6.3.1	Description textuelle des cas d'utilisation	40
6.4	Conception	40
6.4.1	Diagramme de séquence : « Pointer un employé »	40
6.4.2	Diagramme de séquence : « Valider une demande »	40
6.4.3	Diagramme d'états-transitions : cycle de vie d'une demande	43
6.4.4	Diagramme d'activité : traitement d'une demande	43
6.5	Réalisation : interfaces utilisateur	45
7	Sprint 5 : Chaîne ETL et volet décisionnel	46
7.1	Objectifs du Sprint	46
7.2	Spécification des besoins	46
7.2.1	Diagramme de cas d'utilisation du Sprint 5	46
7.2.2	Sprint Backlog du sprint 5	46
7.3	Analyse des besoins	48
7.3.1	Description textuelle des cas d'utilisation	48
7.4	Conception	48
7.4.1	Chaîne décisionnelle et processus ETL	48
7.4.2	Modèle en étoile de l'entrepôt	50
7.5	Réalisation	51
7.5.1	Mise en œuvre de la chaîne ETL en Python	51
7.5.2	Tableau de bord et restitution Power BI	51

8	Sprint 6 : Segmentation des agences par apprentissage non supervisé	53
8.1	Objectifs du Sprint	53
8.2	Spécification des besoins	53
8.2.1	Diagramme de cas d'utilisation du Sprint 6	53
8.2.2	Sprint Backlog du sprint 6	53
8.3	Analyse des besoins	54
8.3.1	Description textuelle du cas d'utilisation	54
8.4	Conception : démarche de clustering	55
8.4.1	Données et variables	55
8.4.2	Prétraitement	55
8.4.3	Détermination du nombre de clusters	55
8.4.4	Comparaison des modèles de clustering	56
8.5	Réalisation : résultats et interprétation	57
8.5.1	Apport décisionnel	58
	Conclusion générale et perspectives	59
	Nétographie	60
A	Annexes	61
A.1	Scripts SQL d'initialisation	61
A.2	Captures complémentaires	61
A.3	Dépôt du code source	61

Table des figures

1.1	Logo de la BTK – Banque Tuniso-Koweïtienne	3
2.1	Diagramme de cas d'utilisation global	12
2.2	Diagramme de classes du modèle métier	15
2.3	Planification du projet (diagramme de Gantt des sprints)	16
2.4	Logo de « $\text{\LaTeX}$ »	16
2.5	Logo de « Java (JDK 21) »	17
2.6	Logo de « Jakarta EE »	17
2.7	Logo de « Hibernate »	17
2.8	Logo d'« Oracle Database »	17
2.9	Logo de « WildFly / JBoss »	17
2.10	Logo d'« Apache Maven »	18
2.11	Logo d'« IntelliJ IDEA »	18
2.12	Logos de « Git » et « GitHub »	18
2.13	Logo d'« ApexCharts »	18
2.14	Logo de « Microsoft Power BI »	18
2.15	Logos de « Python », « pandas » et « scikit-learn »	19
2.16	Architecture globale du système	20
2.17	Architecture physique (diagramme de déploiement 3-tiers)	21
2.18	Architecture logique en couches de l'application	22
3.1	Diagramme de cas d'utilisation du Sprint 1	24
3.2	Diagramme de séquence – Authentification	26

3.3	Interface d'authentification	27
4.1	Diagramme de cas d'utilisation du Sprint 2	29
4.2	Diagramme de séquence – Création d'une agence	31
4.3	Interface de gestion des agences (sélection et consultation des membres) . . .	32
4.4	Interface de gestion des employés (ajout, filtres et liste)	32
5.1	Diagramme de cas d'utilisation du Sprint 3	34
5.2	Diagramme de séquence – Gestion d'un client	36
5.3	Interface de gestion des clients (portefeuille BTK)	36
5.4	Suivi des objectifs commerciaux par agence et gestionnaire	37
6.1	Diagramme de cas d'utilisation du Sprint 4	40
6.2	Diagramme de séquence – Pointage d'un employé	42
6.3	Diagramme de séquence – Validation d'une demande	42
6.4	Diagramme d'états-transitions – Cycle de vie d'une demande	43
6.5	Diagramme d'activité – Traitement d'une demande	44
6.6	Interface du module de pointage des employés	45
7.1	Diagramme de cas d'utilisation du Sprint 5	47
7.2	Chaîne décisionnelle : du système opérationnel au datamart et à sa restitution	49
7.3	Modèle décisionnel en étoile (vues V_BI_*)	50
7.4	Tableau de bord décisionnel embarqué de l'application	52
8.1	Diagramme de cas d'utilisation du Sprint 6	54
8.2	Choix du nombre de clusters : méthode du coude et score de silhouette	56
8.3	Comparaison des modèles selon le score de silhouette	57
8.4	Segmentation des agences (K-Means) projetée par PCA	57

Liste des tableaux

1.1	Fiche signalétique de la BTK	3
1.2	Méthodes traditionnelles vs méthodes agiles	6
1.3	Kanban vs Scrum	7
2.1	Identification des acteurs	9
2.2	Besoins non fonctionnels	9
2.3	Répartition des rôles Scrum du projet	10
2.4	Backlog produit de l'application	11
2.5	Planification des sprints	16
3.1	Sprint Backlog du sprint 1	24
3.2	Description textuelle du cas d'utilisation « S'authentifier »	25
3.3	Description textuelle du cas d'utilisation « Gérer les rôles et les droits d'accès »	25
4.1	Sprint Backlog du sprint 2	29
4.2	Description textuelle du cas d'utilisation « Gérer les agences »	30
4.3	Description textuelle du cas d'utilisation « Gérer les employés »	30
5.1	Sprint Backlog du sprint 3	34
5.2	Description textuelle du cas d'utilisation « Gérer les clients »	35
5.3	Description textuelle du cas d'utilisation « Suivre les objectifs commerciaux »	35
6.1	Sprint Backlog du sprint 4	39
6.2	Description textuelle du cas d'utilisation « Pointer la présence »	41
6.3	Description textuelle du cas d'utilisation « Valider une demande »	41

7.1	Sprint Backlog du sprint 5	47
7.2	Description textuelle du cas d'utilisation « Alimenter le datamart (ETL) »	48
7.3	Description textuelle du cas d'utilisation « Consulter le tableau de bord » . . .	48
7.4	Indicateurs de performance (KPI) du volet décisionnel	50
8.1	Sprint Backlog du sprint 6	54
8.2	Description textuelle du cas d'utilisation « Segmenter les agences »	54
8.3	Variables de segmentation (par agence)	55
8.4	Comparaison des modèles de clustering	56
8.5	Profil moyen des segments d'agences	58

Liste des acronymes

Acronyme	Signification
BTk	Banque Tuniso-Koweïtienne
BI	Business Intelligence (informatique décisionnelle)
ETL	Extract, Transform, Load
IA	Intelligence Artificielle
ML	Machine Learning (apprentissage automatique)
ACP	Analyse en Composantes Principales (PCA)
ORM	Object-Relational Mapping
KPI	Key Performance Indicator (indicateur clé de performance)
PFE	Projet de Fin d'Études
UML	Unified Modeling Language
JPA	Jakarta Persistence API
EJB	Enterprise Java Beans
JSP	Jakarta Server Pages
JSTL	Jakarta Standard Tag Library
JTA	Jakarta Transaction API
JDBC	Java Database Connectivity
MVC	Modèle-Vue-Contrôleur
SGBD	Système de Gestion de Base de Données
WAR	Web Application Archive

Acronyme	Signification
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol

Introduction générale

La transformation numérique du secteur bancaire impose aux établissements de disposer de systèmes d'information capables, d'une part, de centraliser la gestion de leur réseau d'agences et, d'autre part, d'exploiter leurs données pour piloter l'activité. La maîtrise de l'information — effectifs, portefeuille clients, objectifs commerciaux — constitue aujourd'hui un levier déterminant de performance et d'aide à la décision.

C'est dans ce contexte que s'inscrit ce projet de fin d'études, réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne** en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'Esprit School of Business. Le projet consiste à concevoir et réaliser une application web de gestion des agences bancaires, adossée à un volet décisionnel destiné à l'aide à la prise de décision.

L'application couvre la gestion des agences, des employés, des clients et des objectifs commerciaux, le **pointage (présence) des employés**, ainsi qu'un ensemble de circuits de validation (workflows) encadrant la création et la modification des données sensibles. À ces fonctionnalités transactionnelles s'ajoute un volet décisionnel : une chaîne **ETL** consolide les données dans un datamart qui alimente des rapports Microsoft Power BI, un tableau de bord embarqué et une **segmentation des agences** par apprentissage automatique.

Conduit selon la méthode agile **Scrum**, ce rapport est organisé comme suit. Le **premier chapitre** présente le cadre général du projet : l'organisme d'accueil, l'étude de l'existant, la problématique et la méthodologie. Le **deuxième chapitre** (Sprint 0) est consacré à l'analyse et à la spécification des besoins, à la planification des sprints, à l'environnement de développement et à l'architecture du système. Les **chapitres suivants** détaillent les six sprints de réalisation : authentification et gestion des rôles ; gestion des agences et des employés ; gestion des clients et des objectifs ; pointage, demandes et notifications ; chaîne ETL et volet décisionnel ; segmentation des agences. Une conclusion générale dresse le bilan du travail et ouvre des perspectives d'évolution.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre situe le projet dans son contexte. Il délimite d'abord le cadre du projet, présente ensuite l'organisme d'accueil et ses missions, puis analyse la situation existante afin d'en dégager la problématique et la solution proposée. Il se termine par la présentation de la méthodologie de gestion de projet retenue, le cadre agile **Scrum**.

1.1 Cadre du projet

Le présent travail constitue un **projet de fin d'études** réalisé en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'**Esprit School of Business**. Il s'est déroulé dans le cadre d'un stage au sein de la **BTK – Banque Tuniso-Koweïtienne**, du **16 février au 16 juin 2026**. Le projet porte sur la conception et le développement d'une application web de gestion du réseau d'agences bancaires, adossée à un volet décisionnel d'aide à la décision.

1.2 Présentation de l'organisme d'accueil

La **Banque Tuniso-Koweïtienne** (BTK Bank) est une banque universelle de droit tunisien, créée le 25 février 1981 dans le cadre d'une convention de coopération bilatérale entre la Tunisie et le Koweït. Constituée en société anonyme et régie par la loi bancaire n° 2016-48, elle dispose d'un capital social de 200 millions de dinars et a son siège social à Tunis. À travers un réseau

d'agences couvrant l'ensemble du territoire tunisien, elle accompagne aussi bien les particuliers que les entreprises.



Figure 1.1 – Logo de la BTK – Banque Tuniso-Koweïtienne

Le tableau 1.1 en résume la fiche signalétique.

Table 1.1 – Fiche signalétique de la BTK

Élément	Renseignement
Dénomination sociale	Banque Tuniso-Koweïtienne (BTK Bank)
Forme juridique	Société Anonyme (S.A.)
Date de création	25 février 1981
Capital social	200 000 000 DT
Siège social	10 bis, Avenue Mohamed V, 1001 Tunis
Secteur d'activité	Banque universelle
Actionnaire de référence	Groupe Elloumi (60 %)
Réseau	5 directions régionales, 6 succursales, 45 agences
Effectif (2024)	≈ 400 collaborateurs

1.2.1 Missions

En tant que banque universelle, la BTK assure un ensemble de missions couvrant les principaux métiers bancaires :

- **Banque de détail** : comptes, cartes, crédits à la consommation et immobiliers, épargne et banque digitale destinés aux particuliers ;
- **Banque des entreprises** : financement de l'investissement et du fonctionnement, financement du commerce extérieur et gestion de trésorerie ;
- **Opérations internationales et de marché** : change, couverture du risque de change et placements ;
- **Innovation et digitalisation** : modernisation des services et offre en ligne.

1.2.2 Objectifs

À travers ces missions, la BTK poursuit plusieurs objectifs stratégiques :

- soutenir le tissu économique en finançant les entreprises et les projets d'investissement ;
- proposer des solutions financières adaptées aux ménages ;
- contribuer à la création d'emplois et à la dynamique de croissance nationale ;
- consolider sa solidité financière et accélérer sa transformation digitale.

1.3 Étude de l'existant

1.3.1 Description de la situation existante

La gestion actuelle du réseau repose en grande partie sur des traitements manuels et des fichiers bureautiques non centralisés. Les informations relatives aux agences, aux employés et aux clients sont saisies et conservées de manière hétérogène ; les demandes de création ou de modification circulent par des canaux informels ; et la présence des employés n'est suivie par aucun dispositif formalisé.

1.3.2 Critique de la situation existante

Cette organisation présente plusieurs limites :

- dispersion et redondance des données, sources d'incohérences ;
- absence de circuit de validation formalisé pour les opérations sensibles ;
- absence de suivi de présence (pointage) des employés ;
- faible traçabilité des actions et des décisions ;
- difficulté à produire des indicateurs consolidés pour le pilotage ;
- gestion des droits d'accès insuffisamment maîtrisée.

1.3.3 Problématique

La principale problématique qui se pose est la suivante : comment **centraliser et fiabiliser** la gestion du réseau d'agences bancaires, **encadrer** les opérations sensibles par des circuits de

validation, **assurer le suivi de présence** des employés, et **exploiter** les données ainsi structurées pour fournir aux responsables des indicateurs d'aide à la décision ?

1.3.4 Solution proposée

La solution retenue consiste en une **application web centralisée**, structurée autour d'une base de données Oracle, qui couvre la gestion des agences, des employés, des clients et des objectifs, gère les rôles et la sécurité des accès, assure le **pointage** de présence des employés, et formalise les demandes de création et de modification au moyen de **workflows de validation**. Un **volet décisionnel** complète le dispositif : une chaîne ETL consolide les données dans un datamart qui alimente à la fois Microsoft Power BI, un tableau de bord embarqué et une **segmentation des agences** par apprentissage automatique.

1.3.5 Justification du choix de la solution

Le choix d'une application web centralisée se justifie par plusieurs facteurs : l'**accessibilité** (un simple navigateur suffit, sans installation sur les postes), la **centralisation** des données dans une base unique garantissant leur cohérence, la **sécurité** offerte par une gestion fine des rôles et des workflows de validation, et l'**ouverture décisionnelle** permise par l'exposition des données à des outils d'analyse et de *reporting*.

1.4 Méthodologie adoptée

1.4.1 Méthodes traditionnelles et leurs limites

Les méthodes classiques (cycle en cascade, cycle en V) reposent sur un enchaînement **séquentiel** de phases (analyse, conception, développement, tests) planifiées dès le départ. Elles conviennent aux projets dont les besoins sont stables et parfaitement connus à l'avance, mais montrent vite leurs limites dès que le périmètre évolue : faible flexibilité, coût élevé des changements, livraison tardive et détection des risques en fin de cycle.

1.4.2 Origines et apport des méthodes agiles

En réponse à ces limites, les **méthodes agiles** privilégient une approche **itérative et incrémentale**, fondée sur la collaboration continue avec le client, l'adaptation au changement et la livraison fréquente d'incréments fonctionnels. Le tableau 1.2 oppose les deux familles.

Table 1.2 – Méthodes traditionnelles vs méthodes agiles

Critère	Méthodes traditionnelles	Méthodes agiles
Approche	Séquentielle et planifiée (cascade, cycle en V)	Itérative et incrémentale
Flexibilité	Faible : les changements sont coûteux	Élevée : adaptation continue
Livraison	En fin de projet	Par incréments fréquents
Implication du client	Au début et à la fin	Tout au long du projet
Documentation	Abondante et préalable	Allégée, au juste nécessaire
Détection des risques	Tardive	Précoce

1.4.3 Comparaison des cadres agiles : Kanban et Scrum

Parmi les cadres agiles, deux sont particulièrement répandus. **Kanban** repose sur la visualisation d'un *flux continu* de tâches et la limitation du travail en cours, sans rôles ni itérations imposés ; il convient surtout aux activités de maintenance. **Scrum** organise le travail en *itérations de durée fixe* (les sprints), avec des rôles, des artefacts et des cérémonies définis. Le tableau 1.3 les compare.

1.4.4 Choix de la méthodologie : Scrum

Le projet visant à développer un ensemble de fonctionnalités planifiées et livrables par incréments, le cadre **Scrum** a été retenu. Le périmètre ayant été précisé progressivement au fil des échanges avec l'organisme d'accueil, ses sprints, ses rôles et ses cérémonies offrent une structure adaptée au suivi régulier de l'avancement et à la validation progressive des incréments.

Table 1.3 – Kanban vs Scrum

Critère	Kanban	Scrum
Cadence	Flux continu	Sprints de durée fixe
Rôles	Non imposés	Product Owner, Scrum Master, équipe
Cérémonies	Optionnelles	Planning, mêlée, revue, rétrospective
Gestion du travail	Tableau Kanban, limite du WIP	Sprint backlog
Changements	À tout moment	Figés pendant le sprint
Adapté à	Maintenance, flux continu	Projets à fonctionnalités planifiées

1.4.5 Présentation de Scrum

Scrum est un cadre de travail agile reposant sur trois **rôles**, des **artefacts** et des **cérémonies** :

- **Rôles** : le *Product Owner* porte la vision produit et priorise le backlog ; le *Scrum Master* veille au respect de la démarche ; l'*équipe de développement* réalise les incréments ;
- **Artefacts** : le *product backlog* (liste priorisée des besoins), le *sprint backlog* (besoins sélectionnés pour le sprint) et l'*incrément* (le produit potentiellement livrable) ;
- **Cérémonies** : la planification du sprint (*sprint planning*), la mêlée quotidienne (*daily scrum*), la revue de sprint (*sprint review*) et la rétrospective.

Chaque **sprint** est une itération de durée fixe au terme de laquelle un incrément fonctionnel est livré, puis évalué avec le Product Owner avant d'enchaîner sur le sprint suivant.

Conclusion

Ce chapitre a posé le cadre du projet : l'organisme d'accueil et ses missions, l'étude de l'existant qui a mis en évidence la problématique, la solution proposée et la méthodologie Scrum retenue. Le chapitre suivant, consacré au *Sprint 0*, détaille l'analyse et la spécification des besoins, la planification des sprints, l'environnement de développement et l'architecture du système.

Chapitre 2

Sprint 0 : Analyse et Spécification des Besoins

Introduction

Ce chapitre constitue le *Sprint 0*, phase de cadrage qui précède les sprints de réalisation. Il identifie les acteurs et spécifie les besoins fonctionnels et non fonctionnels, formalise le pilotage Scrum et le *backlog* produit, présente les diagrammes de cas d'utilisation global et de classes, planifie les sprints, puis décrit l'environnement de développement et l'architecture du système.

2.1 Étude du contexte

2.1.1 Identification des acteurs

Trois profils interagissent avec l'application, conformément aux rôles gérés par le module de sécurité (table `SECURITY_USERS`). Le tableau 2.1 en précise les responsabilités.

2.1.2 Spécification des besoins

2.1.2.1 Besoins fonctionnels

Le système doit permettre :

- l'authentification sécurisée et la gestion des rôles et des droits d'accès (ADMIN, DIRECTEUR_COMMERCIAL, USER);

Table 2.1 – Identification des acteurs

Acteur	Responsabilités
Administrateur (ADMIN)	Gère les agences, les employés et les clients; valide les demandes de création de clients et d'employés; consulte le journal d'audit; supervise le volet décisionnel.
Directeur commercial (DIRECTEUR_COMMERCIAL)	Soumet les demandes de création (clients, employés); valide les demandes de modification des données des employés; consulte les objectifs et les indicateurs.
Utilisateur (USER)	Consulte les données autorisées, effectue son pointage et soumet des demandes de modification de ses propres informations.

- la gestion (CRUD) des agences, des employés et des clients;
- la consultation des objectifs commerciaux et le suivi de la production;
- le **pointage (présence) des employés** : saisie de l'heure d'arrivée et de départ, statut (présent / retard / absent), en mode automatique (bouton) ou manuel (administrateur);
- la soumission et la validation des demandes (clients, employés, modifications) selon le cycle EN_ATTENTE → EN_COURS → EFFECTUE/REFUSE;
- la notification des demandes en attente et la journalisation des actions;
- la consolidation des données par une chaîne **ETL** et la restitution d'indicateurs via un tableau de bord et Power BI;
- la **segmentation des agences** par apprentissage non supervisé.

2.1.2.2 Besoins non fonctionnels

Le tableau 2.2 récapitule les besoins non fonctionnels.

Table 2.2 – Besoins non fonctionnels

Besoin	Description
Sécurité	Authentification, gestion des rôles et protection des opérations sensibles par des workflows de validation.
Fiabilité	Intégrité des données garantie par le SGBD et les transactions JTA.
Performance	Temps de réponse acceptable sur les volumes manipulés.
Ergonomie	Interface claire et cohérente, navigation par menu latéral.
Maintenabilité	Architecture en couches, code modulaire et faiblement couplé.
Portabilité	Application Jakarta EE standard, déployable sur serveur compatible.

2.2 Pilotage du projet avec Scrum

Le pilotage du projet repose sur la méthode **Scrum** présentée au chapitre précédent. Le tableau 2.3 précise la répartition des rôles adoptée durant le stage.

Table 2.3 – Répartition des rôles Scrum du projet

Rôle	Assuré par	Mission
Product Owner	Maître de stage (BTK)	Exprime et priorise les besoins, valide les fonctionnalités livrées.
Scrum Master	Encadrant acadé- mique	Veille au respect de la démarche Scrum et au bon déroulement des itérations.
Équipe de développement	Étudiant	Prend en charge l'analyse, la conception, le développement et les tests.

2.3 Backlog du produit

Le *product backlog* constitue la liste ordonnée et évolutive des besoins, exprimés sous forme de *user stories* priorisées selon leur valeur métier. Il guide la planification des sprints et sert de source unique de travail. Le tableau 2.4 en présente le contenu.

2.4 Diagramme de cas d'utilisation global

La figure 2.1 présente le **diagramme de cas d'utilisation global**, qui représente d'un point de vue fonctionnel les interactions entre les acteurs et les services offerts par le système.

Table 2.4 – Backlog produit de l'application

ID	Fonctionnalité	US	User story	Priorité
01	Authentification et sécurité	US1	S'authentifier avec identifiant et mot de passe.	Haute
		US2	Gérer les rôles et les droits d'accès.	Haute
02	Gestion des agences	US3	Gérer les agences (créer, consulter, modifier, supprimer).	Haute
03	Gestion des employés	US4	Gérer les employés et leur rattachement aux agences.	Haute
		US5	Gérer les gestionnaires et leurs portefeuilles.	Moyenne
04	Gestion des clients	US6	Gérer les clients (portefeuille).	Moyenne
		US7	Affecter un client à un gestionnaire.	Moyenne
05	Objectifs commerciaux	US8	Suivre les objectifs commerciaux et la production.	Moyenne
06	Pointage	US9	Pointer la présence (arrivée / départ).	Haute
		US10	Saisir ou corriger un pointage manuel.	Moyenne
07	Workflow de demandes	US11	Soumettre une demande (client, employé, modification).	Moyenne
		US12	Valider ou refuser une demande.	Moyenne
		US13	Recevoir des notifications et consulter le journal d'audit.	Moyenne
08	Volet décisionnel	US14	Alimenter le datamart via la chaîne ETL.	Haute
		US15	Consulter le tableau de bord décisionnel.	Haute
		US16	Consulter les rapports Power BI.	Moyenne
09	Segmentation (IA)	US17	Segmenter les agences par apprentissage non supervisé.	Moyenne

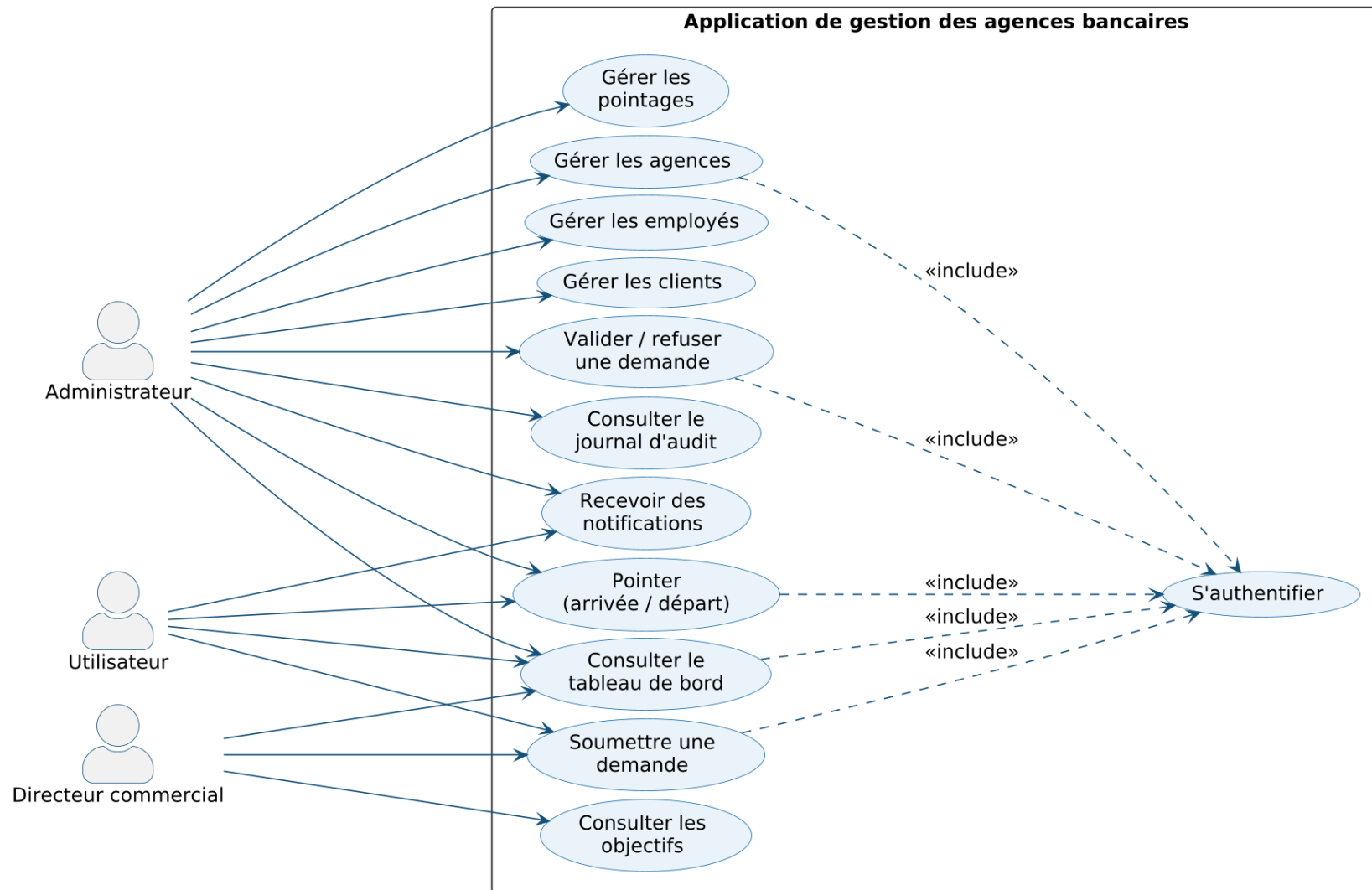


Figure 2.1 – Diagramme de cas d'utilisation global

On y distingue les trois acteurs et leurs cas d'utilisation respectifs. Le cas *S'authentifier* est commun à tous ; il est relié aux autres par une relation «include», l'accès aux fonctionnalités exigeant une authentification préalable. L'**administrateur** concentre les cas de gestion et de validation ; le **directeur commercial** soumet les demandes et consulte les indicateurs ; l'**utilisateur** se limite à la consultation, au pointage et aux demandes le concernant.

2.5 Diagramme de classe

La figure 2.2 présente le **diagramme de classes** du modèle métier, qui représente les entités persistantes du domaine, leurs attributs et les associations qui les relient.

L'entité *Utilisateur* occupe une position centrale. Une *Agence* regroupe plusieurs *Utilisateur*, *Client* et *Objectif* ; un *Gestionnaire* encadre plusieurs *Client* ; les entités de demande (*DemandeClient*, *DemandeEmploye*, *DemandeModification*, *DemandeAgence*), ainsi que *JournalAction*, *Notification* et *Pointage*, référencent un *Utilisateur* par une association @ManyToOne. Enfin, *SecurityUser* est relié à *Utilisateur* par une dépendance d'authentification. Le modèle ne comporte ni héritage ni composition : les liens sont exclusivement des associations, reflétant fidèlement le code du projet.

2.6 Planification des sprints

Conformément au backlog, le développement a été découpé en **six sprints** de réalisation, précédés du Sprint 0 de cadrage. Le projet s'est déroulé sur quatre mois (16 février – 16 juin 2026). Le tableau 2.5 détaille cette planification et la figure 2.3 en donne le diagramme de Gantt.

2.7 Environnement de développement

2.7.1 Environnement matériel

Le développement a été réalisé sur un poste de travail standard : ordinateur portable doté d'un processeur Intel Core i5, de 16 Go de mémoire vive et du système d'exploitation Win-

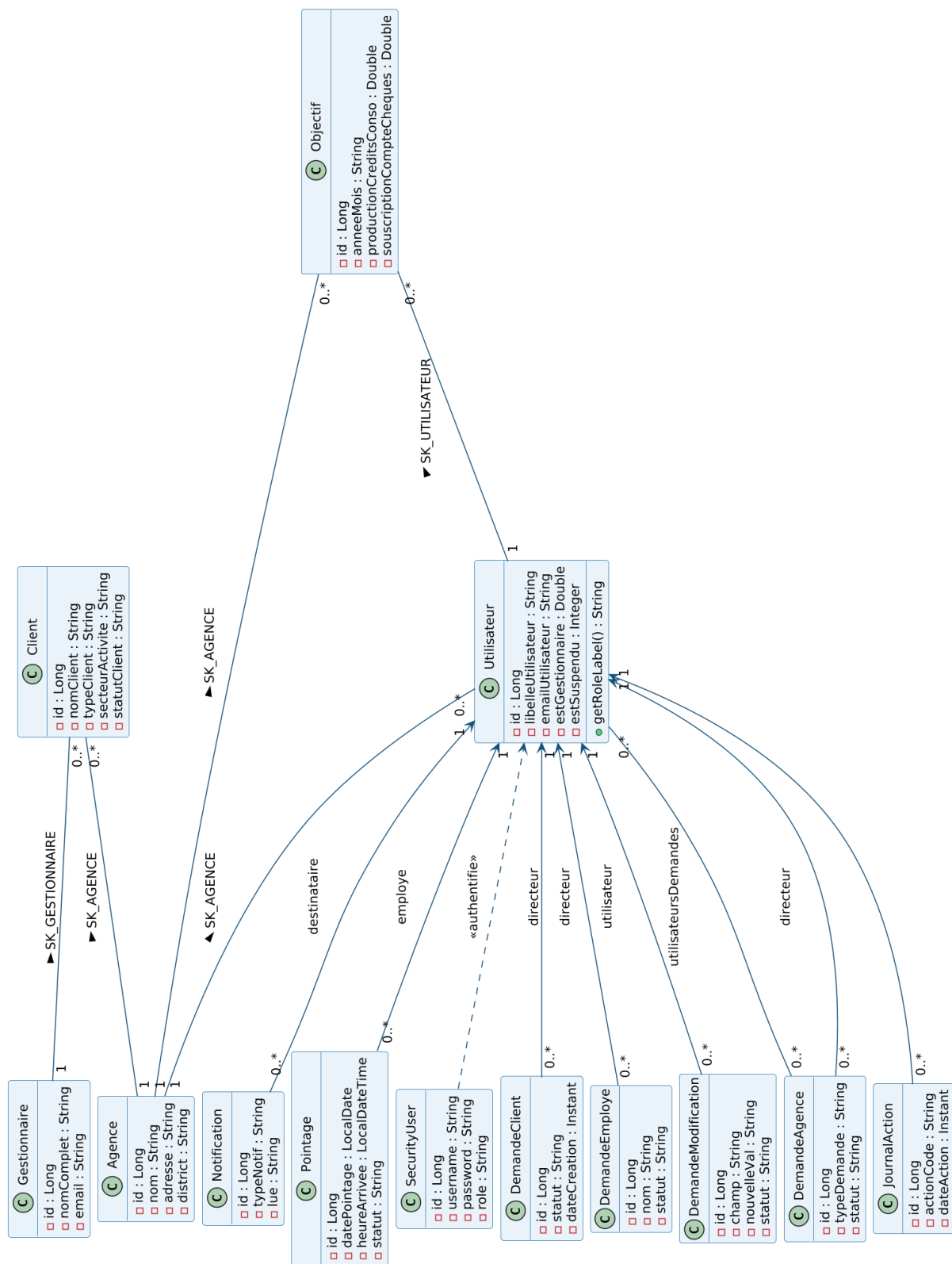


Figure 2.2 – Diagramme de classes du modèle métier

Table 2.5 – Planification des sprints

Sprint	Durée	User stories	Objectif du sprint
Sprint 1	2 semaines	US1, US2	Authentification et gestion des rôles.
Sprint 2	2 semaines	US3, US4, US5	Gestion des agences et des employés.
Sprint 3	2 semaines	US6, US7, US8	Gestion des clients et des objectifs.
Sprint 4	2 semaines	US9–US13	Pointage, demandes et notifications.
Sprint 5	3 semaines	US14, US15, US16	Chaîne ETL et volet décisionnel.
Sprint 6	2 semaines	US17	Segmentation des agences (clustering).

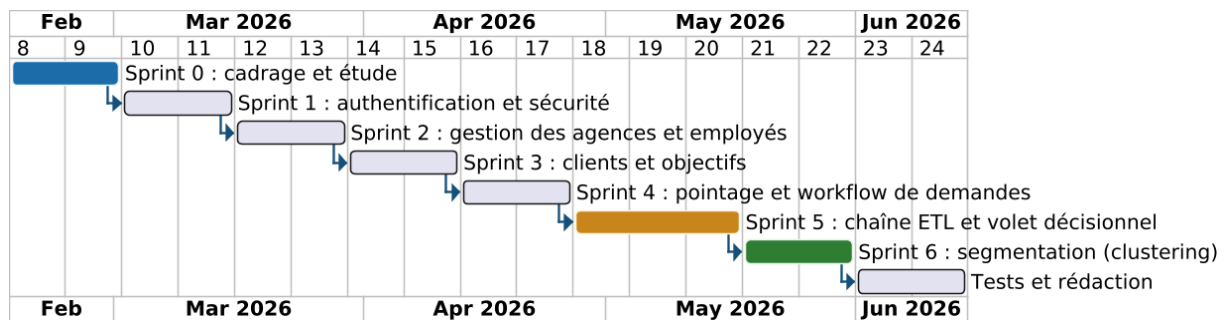


Figure 2.3 – Planification du projet (diagramme de Gantt des sprints)

dows 11.

2.7.2 Outil de rédaction du rapport

Le présent rapport a été rédigé avec **LaTeX**, système de composition de documents particulièrement adapté aux écrits scientifiques et techniques.



Figure 2.4 – Logo de « LaTeX »

2.7.3 Environnement de développement logiciel

Plusieurs outils et technologies ont été mobilisés pour la réalisation du projet.

Java (JDK 21) : langage et plateforme d'exécution sur lesquels repose l'ensemble de l'application.



Figure 2.5 – Logo de « Java (JDK 21) »

Jakarta EE 10 : ensemble de spécifications d'entreprise (Servlets, JSP, EJB, JPA) structurant l'application en couches.



Figure 2.6 – Logo de « Jakarta EE »

Hibernate : implémentation de JPA assurant le mapping objet-relationnel entre les entités Java et la base Oracle.



Figure 2.7 – Logo de « Hibernate »

Oracle Database : système de gestion de base de données relationnelle hébergeant l'ensemble des données opérationnelles et les vues décisionnelles.



Figure 2.8 – Logo d'« Oracle Database »

WildFly / JBoss : serveur d'applications Jakarta EE sur lequel l'application est déployée sous forme d'archive WAR.



Figure 2.9 – Logo de « WildFly / JBoss »

Apache Maven : outil de gestion des dépendances et d'automatisation du *build* (production du WAR).



Figure 2.10 – Logo d'« Apache Maven »

IntelliJ IDEA : environnement de développement intégré utilisé pour le développement, le débogage et le déploiement de l'application.



Figure 2.11 – Logo d'« IntelliJ IDEA »

Git et GitHub : système de gestion de versions et plateforme d'hébergement du code source.



Figure 2.12 – Logos de « Git » et « GitHub »

ApexCharts : bibliothèque JavaScript de visualisation utilisée pour les graphiques du tableau de bord embarqué.



Figure 2.13 – Logo d'« ApexCharts »

Microsoft Power BI : outil de *reporting* décisionnel connecté aux vues analytiques de la base Oracle.



Figure 2.14 – Logo de « Microsoft Power BI »

Python : langage utilisé pour la chaîne **ETL** et la **segmentation** des agences, au moyen des bibliothèques **pandas** (manipulation des données) et **scikit-learn** (apprentissage automatique).



Figure 2.15 – Logos de « Python », « pandas » et « scikit-learn »

2.8 Architecture du système

L'architecture définit l'organisation des composants du système et la manière dont ils communiquent. Elle est présentée à trois niveaux : globale, physique et logique.

2.8.1 Architecture globale

L'architecture globale (figure 2.16) offre une vue d'ensemble des composants. Un poste client (navigateur) dialogue en HTTP/HTTPS avec l'application web déployée sur **WildFly**, structurée en couches (présentation, contrôle, métier, persistance) ; celle-ci accède à la base **Oracle** en JDBC. La base alimente, d'une part, **Power BI** via les vues `V_BI_*` et, d'autre part, la **chaîne décisionnelle Python** (ETL puis segmentation).

2.8.2 Architecture physique

L'architecture physique (figure 2.17) représente le **déploiement** des composants sur les nœuds matériels selon une organisation **3-tiers** : le poste client, le serveur d'applications WildFly hébergeant `gestion-agences.war` (accès à la base via la source de données JTA OracleDS), le serveur de base de données Oracle (`FREEPDB1`) et le poste décisionnel exécutant Power BI.

2.8.3 Architecture logique

L'architecture logique (figure 2.18) décompose le système en **couches** faiblement couplées suivant le patron **MVC** : la couche *présentation* (JSP/JSTL, ApexCharts), la couche *contrôle* (Servlets et le filtre `NotificationFilter`), la couche *métier* (services EJB) et la couche *persistance* (entités JPA et Hibernate dialoguant avec Oracle via l'`EntityManager`).

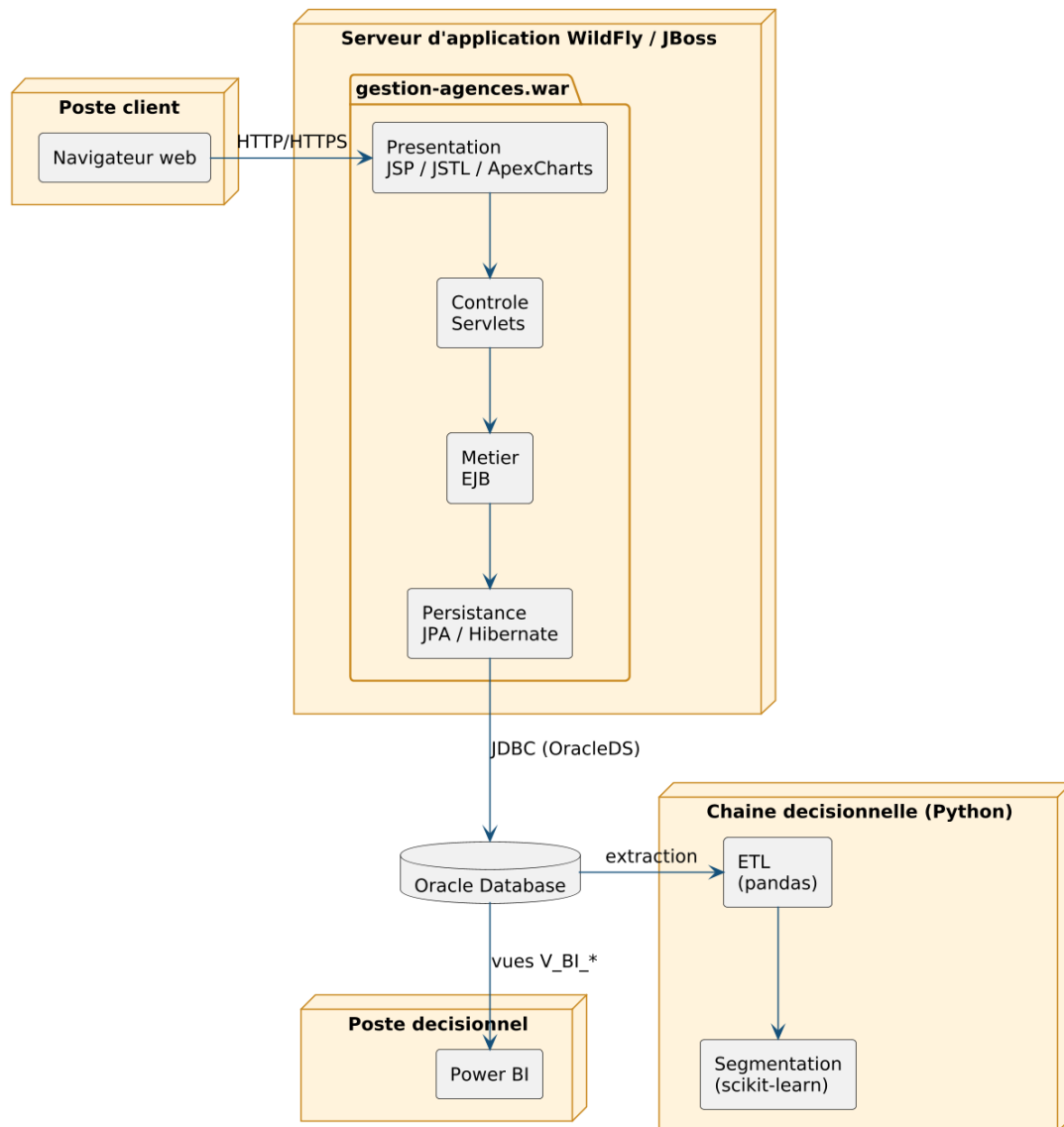


Figure 2.16 – Architecture globale du système

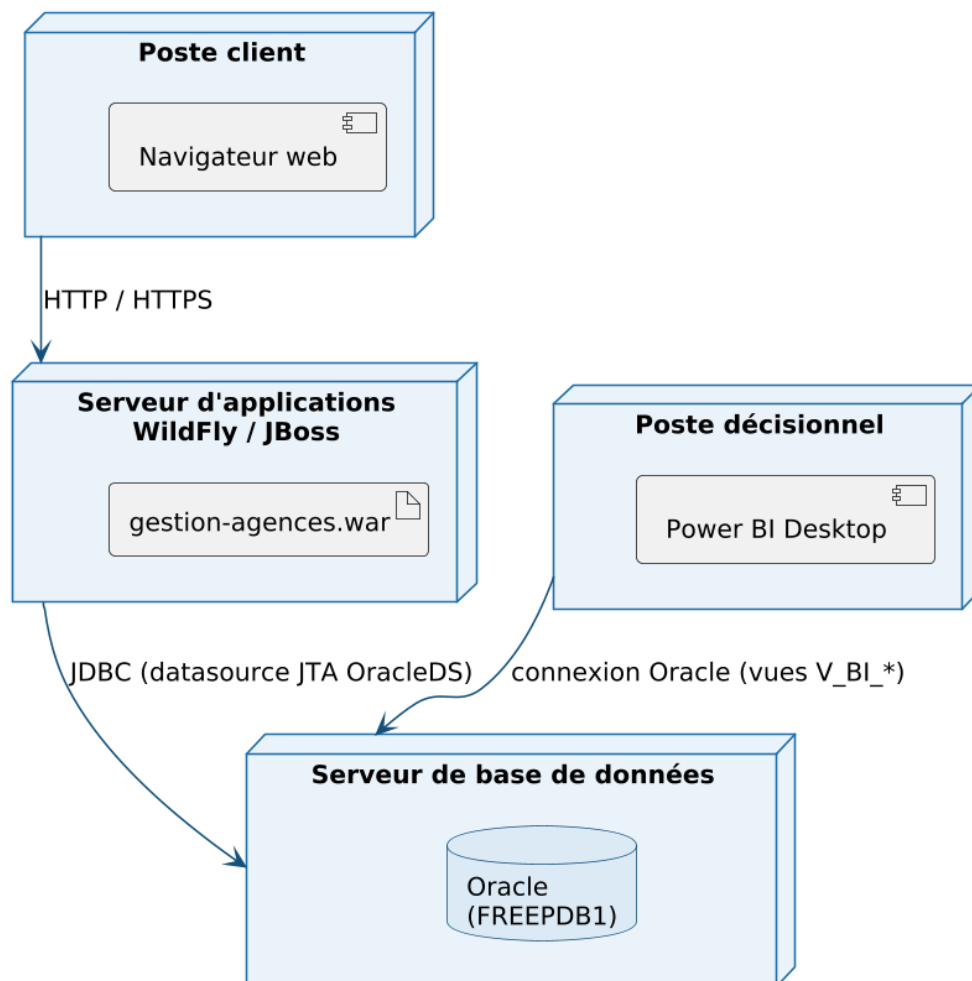
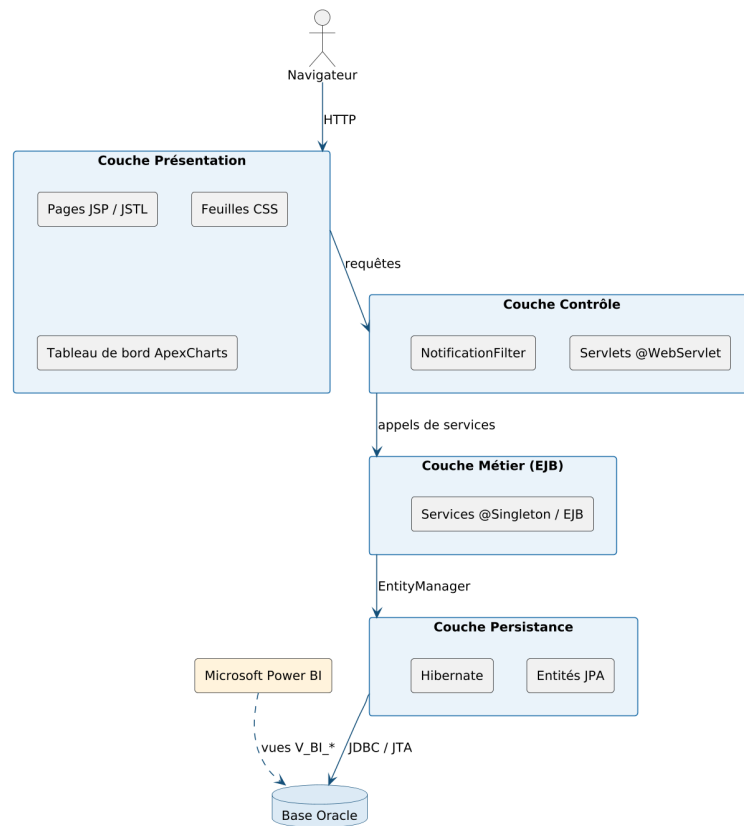


Figure 2.17 – Architecture physique (diagramme de déploiement 3-tiers)



Conclusion

Ce Sprint 0 a posé les fondations du projet : acteurs, besoins, backlog priorisé, planification des sprints, environnement de développement et architecture du système. Les chapitres suivants détaillent chaque sprint de réalisation, en commençant par l'authentification et la gestion des rôles.

Chapitre 3

Sprint 1 : Authentification et gestion des rôles

Introduction

Ce chapitre couvre le premier sprint de réalisation, consacré à la sécurité d'accès. Après un rappel des objectifs, il présente le diagramme de cas d'utilisation et le *sprint backlog*, l'analyse des besoins, la conception au moyen d'un diagramme de séquence, puis les interfaces réalisées.

3.1 Objectifs du Sprint

L'objectif de ce sprint est de mettre en place l'**authentification** des utilisateurs et la **gestion des rôles et des droits d'accès** (ADMIN, DIRECTEUR_COMMERCIAL, USER), socle de sécurité de toute l'application.

3.2 Spécification des besoins

3.2.1 Diagramme de cas d'utilisation du Sprint 1

La figure 3.1 présente les cas d'utilisation du sprint.

3.2.2 Sprint Backlog du sprint 1

Le tableau 3.1 détaille les user stories du sprint et les tâches techniques associées.

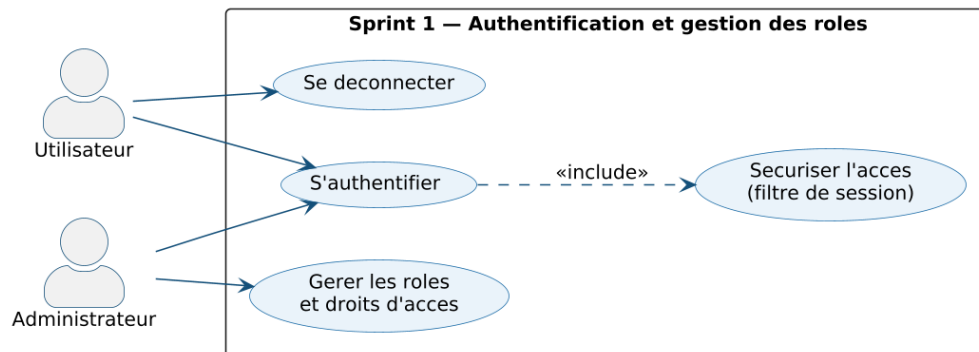


Figure 3.1 – Diagramme de cas d'utilisation du Sprint 1

Table 3.1 – Sprint Backlog du sprint 1

US	User story	Tâches techniques
US1	S'authentifier avec identifiant et mot de passe.	Créer le LoginServlet (POST /login); vérifier les identifiants via le SecurityService sur la table SECURITY_USERS; ouvrir une HttpSession; créer le LogoutServlet.
US2	Gérer les rôles et les droits d'accès.	Associer un rôle (ADMIN, DIRECTEUR_COMMERCIAL, USER) à chaque utilisateur; restreindre l'accès aux fonctionnalités selon le rôle; gérer la réinitialisation du mot de passe (ForgotPasswordServlet).

3.3 Analyse des besoins

3.3.1 Description textuelle des cas d'utilisation

Les tableaux 3.2 et 3.3 décrivent les cas d'utilisation principaux du sprint.

Table 3.2 – Description textuelle du cas d'utilisation « S'authentifier »

Titre	S'authentifier
Objectif	Permettre à l'utilisateur de se connecter à l'application avec son identifiant et son mot de passe.
Acteur	Administrateur / Directeur commercial / Utilisateur
Pré-condition	L'utilisateur doit disposer d'un compte actif dans la table SECURITY_USERS.
Post-condition	Une session est ouverte et l'utilisateur est redirigé vers le tableau de bord correspondant à son rôle.
Scénario nominal	(1) L'utilisateur saisit son identifiant et son mot de passe; (2) le SecurityService vérifie les informations sur la table de sécurité; (3) en cas de succès, une session est ouverte et l'utilisateur est redirigé; (4) sinon, un message d'erreur est affiché.

Table 3.3 – Description textuelle du cas d'utilisation « Gérer les rôles et les droits d'accès »

Titre	Gérer les rôles et les droits d'accès
Objectif	Permettre à l'administrateur d'attribuer un rôle à chaque utilisateur afin de contrôler l'accès aux fonctionnalités.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié avec le rôle ADMIN.
Post-condition	Le rôle est enregistré et les droits d'accès de l'utilisateur sont mis à jour.
Scénario nominal	(1) L'administrateur consulte la liste des utilisateurs; (2) il sélectionne un utilisateur; (3) il lui attribue un rôle (ADMIN, DIRECTEUR_COMMERCIAL ou USER); (4) le système applique le contrôle d'accès correspondant.

3.4 Conception

3.4.1 Diagramme de séquence : « S'authentifier »

La figure 3.2 décrit le déroulement dynamique de l'authentification.

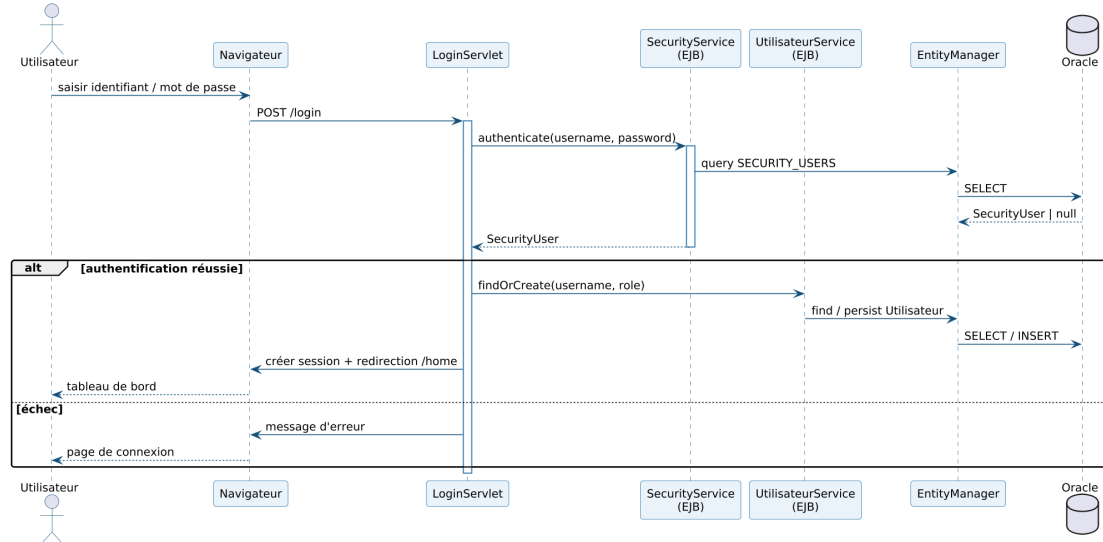


Figure 3.2 – Diagramme de séquence – Authentification

Le LoginServlet reçoit la requête POST /login et délègue la vérification au SecurityService, qui interroge la table SECURITY_USERS via l'EntityManager. Le fragment combiné alt distingue les deux issues : en cas de succès, l'Utilisateur métier est associé (ou créé), une session est ouverte et l'utilisateur est redirigé vers le tableau de bord ; en cas d'échec, un message d'erreur est renvoyé.

3.5 Réalisation : interfaces utilisateur

La figure 3.3 présente la page d'authentification réalisée : l'utilisateur y saisit son identifiant et son mot de passe pour accéder à l'application.

Conclusion

Ce sprint a mis en place le socle de sécurité de l'application : l'authentification et la gestion des rôles. Le sprint suivant aborde la gestion des agences et des employés.

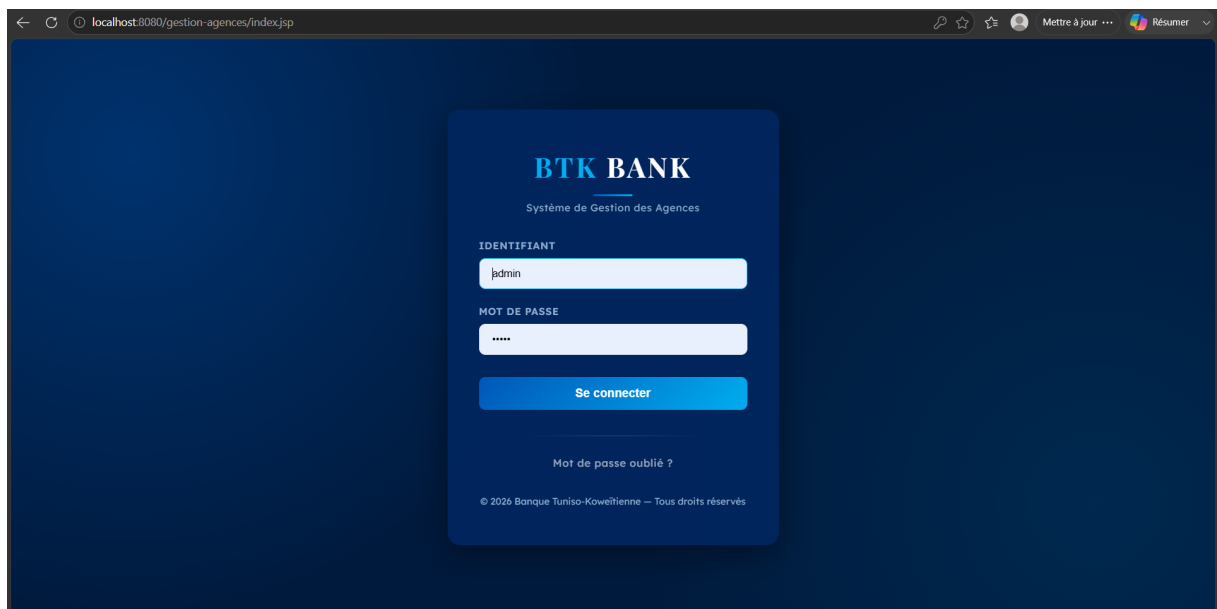


Figure 3.3 – Interface d'authentification

Chapitre 4

Sprint 2 : Gestion des agences et des employés

Introduction

Ce sprint met en place les fonctionnalités de gestion du référentiel : les agences, les employés et leur rattachement. Il suit la même démarche : objectifs, spécification (cas d'utilisation et backlog), analyse, conception et réalisation.

4.1 Objectifs du Sprint

L'objectif est d'offrir la **gestion complète (CRUD)** des agences et des employés, ainsi que le **rattachement** de chaque employé à son agence et la gestion des gestionnaires.

4.2 Spécification des besoins

4.2.1 Diagramme de cas d'utilisation du Sprint 2

La figure 4.1 présente les cas d'utilisation du sprint.

4.2.2 Sprint Backlog du sprint 2

Le tableau 4.1 détaille les user stories et les tâches techniques.

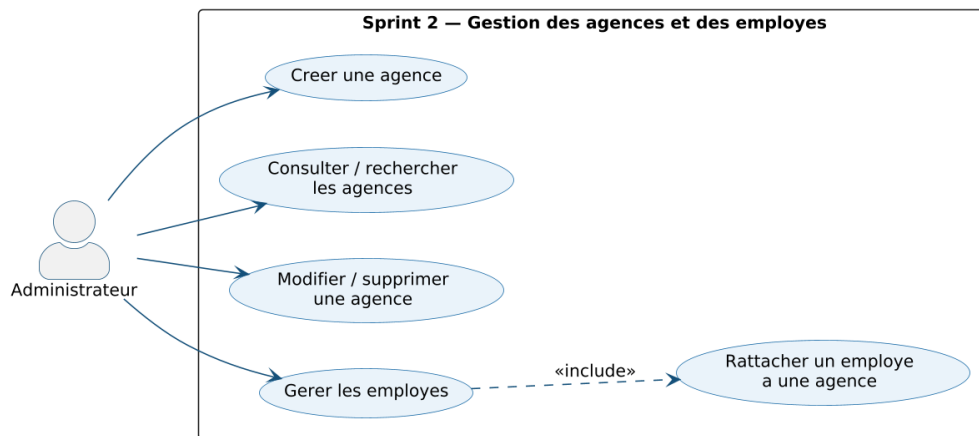


Figure 4.1 – Diagramme de cas d'utilisation du Sprint 2

Table 4.1 – Sprint Backlog du sprint 2

US	User story	Tâches techniques
US3	Gérer les agences (créer, consulter, modifier, supprimer).	Développer l'AgenceServlet et l'AgenceService (EJB) sur l'entité Agence; réaliser la page agences.jsp avec les actions CRUD.
US4	Gérer les employés et leur rattachement aux agences.	Développer l'AjouterEmployeServlet et le BUtilisateurServlet; gérer les employés via l'UtilisateurService; associer chaque employé à une Agence (SK_AGENCE); pages employes.jsp et add-effectif.jsp.
US5	Gérer les gestionnaires et leurs portefeuilles.	Développer le GestionnaireServlet et le GestionnaireService sur l'entité Gestionnaire; page gestionnaire.jsp.

4.3 Analyse des besoins

4.3.1 Description textuelle des cas d'utilisation

Table 4.2 – Description textuelle du cas d'utilisation « Gérer les agences »

Titre	Gérer les agences
Objectif	Permettre à l'administrateur de créer, consulter, modifier et supprimer les agences du réseau.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié avec le rôle ADMIN.
Post-condition	L'agence créée ou modifiée est enregistrée dans la base et la liste est mise à jour.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des agences; (2) il sélectionne une action (créer, modifier, supprimer); (3) il saisit ou met à jour les informations de l'agence; (4) l'AgenceService persiste la modification et la liste est rafraîchie.

Table 4.3 – Description textuelle du cas d'utilisation « Gérer les employés »

Titre	Gérer les employés
Objectif	Permettre à l'administrateur de gérer les employés et de les rattacher à leur agence.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié et l'agence de rattachement doit exister.
Post-condition	L'employé est enregistré et rattaché à son agence.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des employés; (2) il saisit les informations de l'employé; (3) il sélectionne l'agence de rattachement; (4) l'UtilisateurService enregistre l'employé associé à l'agence.

4.4 Conception

4.4.1 Diagramme de séquence : « Créer une agence »

La figure 4.2 décrit le scénario de création d'une agence.

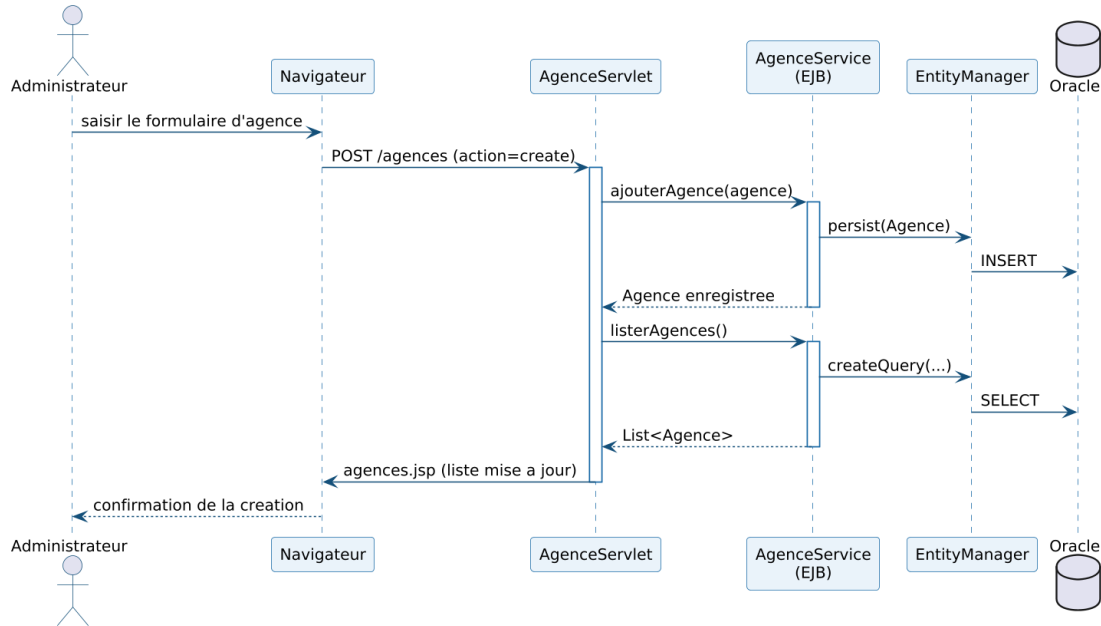


Figure 4.2 – Diagramme de séquence – Création d'une agence

L'AgenceServlet reçoit la requête de création et délègue à l'AgenceService, qui persiste la nouvelle Agence via l'EntityManager (INSERT), puis recharge la liste des agences renvoyée à la page agences.jsp. Le même schéma s'applique aux opérations de modification et de suppression.

4.5 Réalisation : interfaces utilisateur

Les figures 4.3 et 4.4 illustrent les interfaces de gestion des agences et des employés.

Conclusion

Ce sprint a doté l'application de la gestion du référentiel des agences et des employés. Le sprint suivant traite des clients et des objectifs commerciaux.

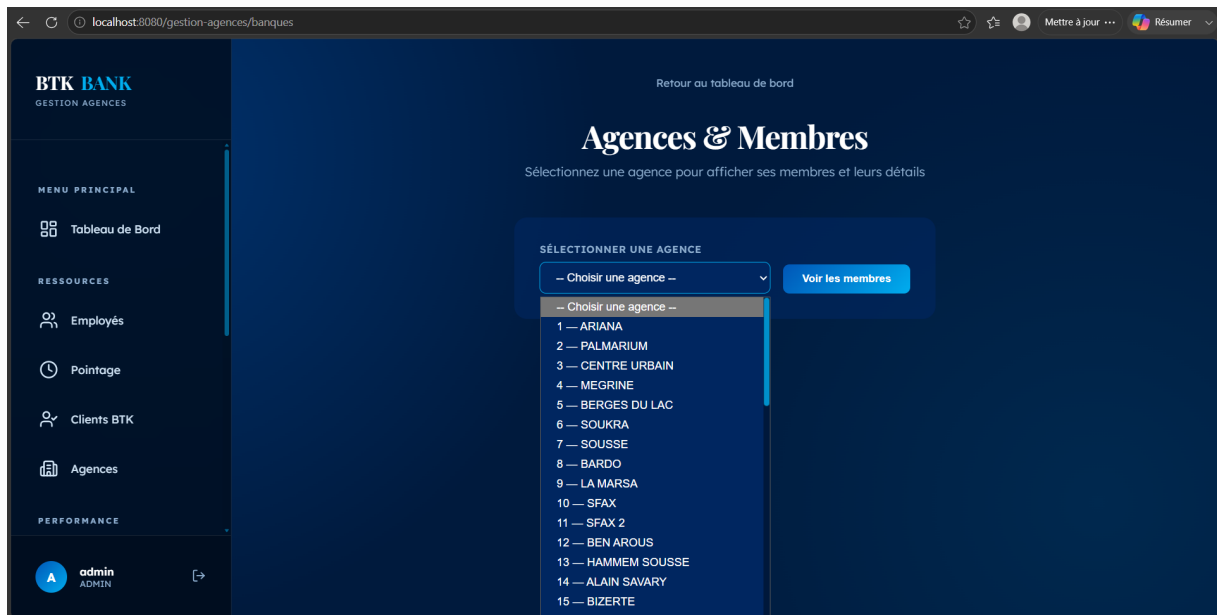


Figure 4.3 – Interface de gestion des agences (sélection et consultation des membres)

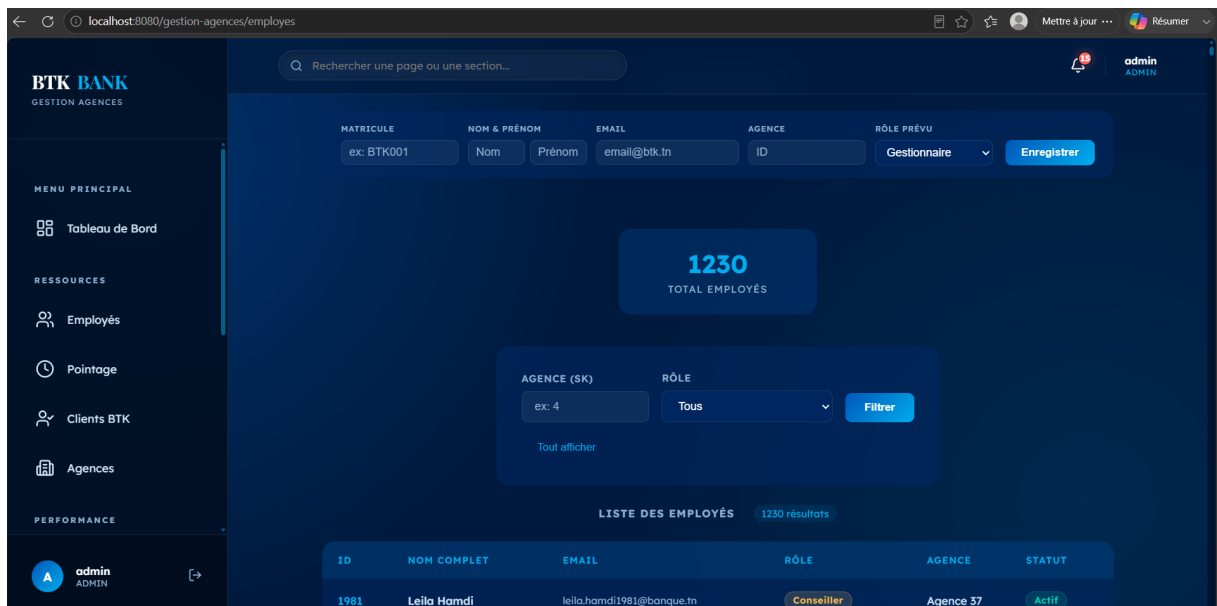


Figure 4.4 – Interface de gestion des employés (ajout, filtres et liste)

Chapitre 5

Sprint 3 : Gestion des clients et des objectifs

Introduction

Ce sprint complète le référentiel commercial : la gestion du portefeuille clients, leur affectation aux gestionnaires et le suivi des objectifs commerciaux.

5.1 Objectifs du Sprint

L'objectif est de permettre la **gestion des clients** (portefeuille), leur **affectation à un gestionnaire** et le **suivi des objectifs commerciaux** (ouvertures de comptes, crédits, épargne) par agence.

5.2 Spécification des besoins

5.2.1 Diagramme de cas d'utilisation du Sprint 3

La figure 5.1 présente les cas d'utilisation du sprint.

5.2.2 Sprint Backlog du sprint 3

Le tableau 5.1 détaille les user stories et les tâches techniques.

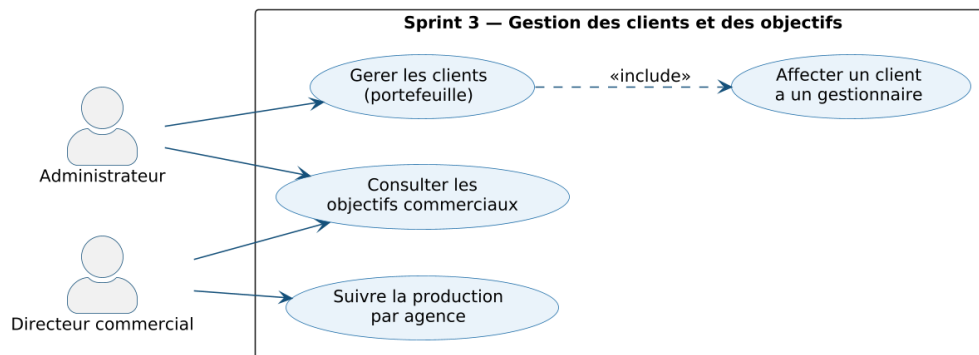


Figure 5.1 – Diagramme de cas d'utilisation du Sprint 3

Table 5.1 – Sprint Backlog du sprint 3

US	User story	Tâches techniques
US6	Gérer les clients (portefeuille).	Développer le <code>ClientServlet</code> et le <code>ClientService</code> sur l'entité <code>Client</code> ; page <code>client.jsp</code> ; gérer les clients BTK (<code>BClientServlet</code> , <code>clients-btk.jsp</code>).
US7	Affecter un client à un gestionnaire.	Associer chaque <code>Client</code> à un <code>Gestionnaire</code> (<code>SK_GESTIONNAIRE</code>); mettre à jour le portefeuille du gestionnaire.
US8	Suivre les objectifs commerciaux et la production.	Développer l' <code>ObjectifServlet</code> et l' <code>ObjectifService</code> sur l'entité <code>Objectif</code> ; pages <code>objectifs.jsp</code> et <code>objectifs-btk.jsp</code> ; restituer comptes, crédits et épargne par agence.

5.3 Analyse des besoins

5.3.1 Description textuelle des cas d'utilisation

Table 5.2 – Description textuelle du cas d'utilisation « Gérer les clients »

Titre	Gérer les clients
Objectif	Permettre à l'administrateur de gérer le portefeuille clients et d'affecter chaque client à un gestionnaire.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié et le gestionnaire de rattachement doit exister.
Post-condition	Le client est enregistré et associé à son gestionnaire dans la base.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des clients ; (2) il saisit ou met à jour les informations du client ; (3) il sélectionne le gestionnaire responsable ; (4) le <code>ClientService</code> enregistre le client et met à jour le portefeuille.

Table 5.3 – Description textuelle du cas d'utilisation « Suivre les objectifs commerciaux »

Titre	Suivre les objectifs commerciaux
Objectif	Permettre au directeur commercial de consulter les objectifs et la production par agence.
Acteur	Directeur commercial / Administrateur
Pré-condition	L'acteur doit être authentifié et des objectifs doivent être renseignés pour la période.
Post-condition	Les indicateurs de production (comptes, crédits, épargne) sont restitués par agence.
Scénario nominal	(1) L'acteur accède à l'interface des objectifs ; (2) il sélectionne une agence et une période ; (3) l' <code>ObjectifService</code> agrège la production ; (4) les indicateurs sont affichés.

5.4 Conception

5.4.1 Diagramme de séquence : « Gérer un client »

La figure 5.2 décrit l'enregistrement et l'affectation d'un client.

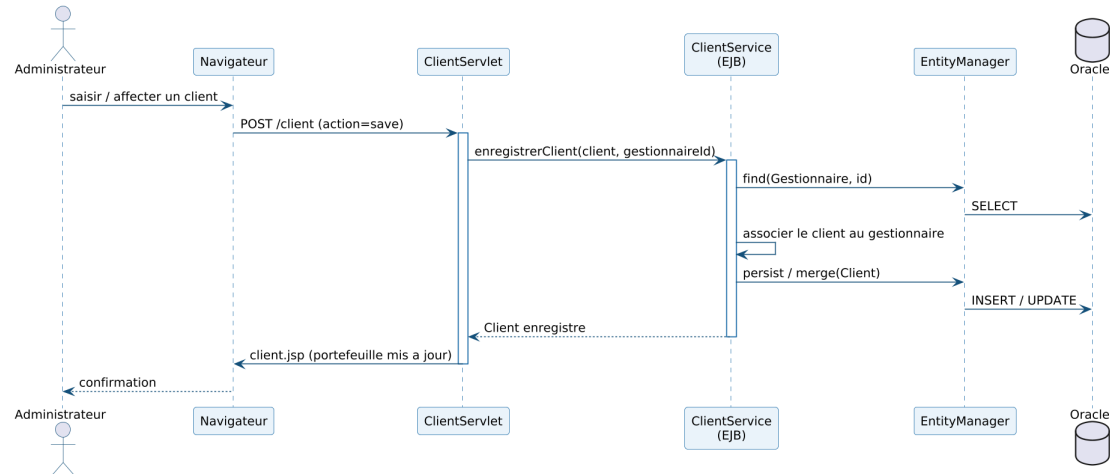


Figure 5.2 – Diagramme de séquence – Gestion d'un client

Le ClientServlet transmet la demande au ClientService, qui recherche le Gestionnaire concerné, associe le Client à ce dernier, puis le persiste via l'EntityManager. Le portefeuille mis à jour est renvoyé à la page client.jsp.

5.5 Réalisation : interfaces utilisateur

Les figures 5.3 et 5.4 illustrent les interfaces de gestion des clients et de suivi des objectifs commerciaux.

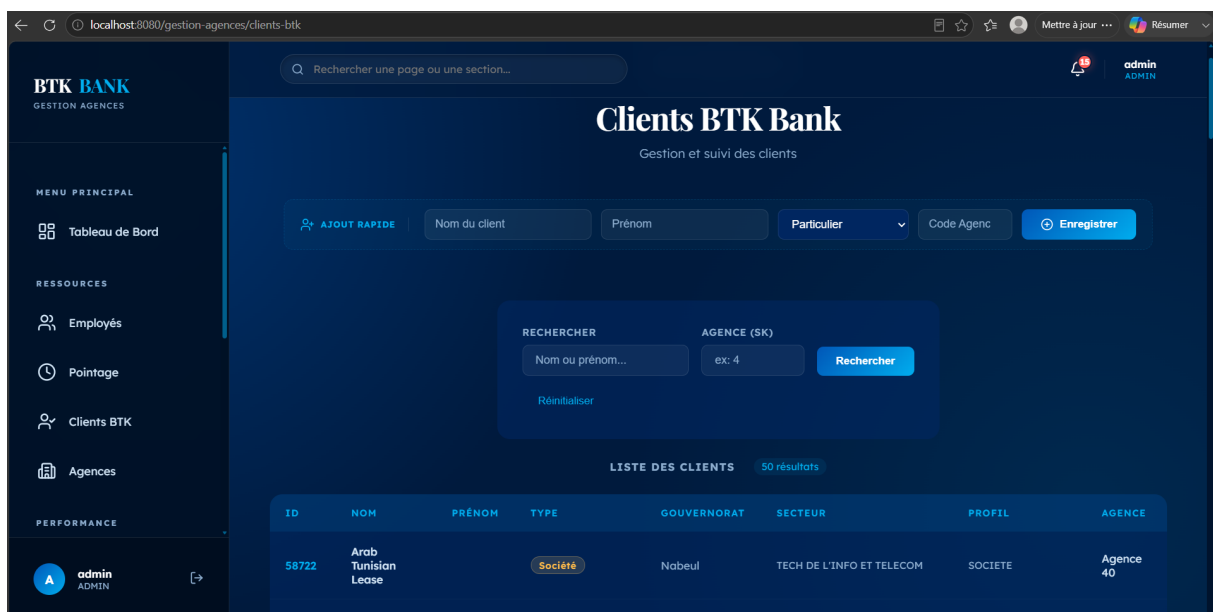


Figure 5.3 – Interface de gestion des clients (portefeuille BTK)

BTK BANK
GESTION AGENCES

Objectifs Commerciaux
Suivi des objectifs par agence et gestionnaire — Total : 376

AGENCE (SK) : ex: 4
GESTIONNAIRE (SK) : ex: 76
Filtrer
Tout afficher

AGENCE	GESTIONNAIRE	SITUATION	EER PART.	EER H.PART.	CPT CHÈQUES	CPT ÉPARGNES	CRÉDITS CONSO	CRÉDITS IMMO	DAV PP
Agence 1	Gest. 67	Détaché	36.0	24.0	27.0	31.0	750000.0	550000.0	144000.0
Agence 1	Gest. 152	CIVP	60.0	24.0	45.0	51.0	900000.0	550000.0	120000.0
Agence 1	Gest. 223	Détaché	96.0	12.0	72.0	82.0	900000.0	550000.0	96000.0
Agence 1	Gest. 307	Permanent	48.0	0.0	36.0	41.0	288000.0	112000.0	48000.0
Agence 1	Gest. 0	Permanent	36.0	0.0	27.0	31.0	0.0	0.0	0.0

admin ADMIN

Figure 5.4 – Suivi des objectifs commerciaux par agence et gestionnaire

Conclusion

Ce sprint a complété la gestion commerciale (clients et objectifs). Le sprint suivant aborde le pointage des employés, les workflows de demandes et les notifications.

Chapitre 6

Sprint 4 : Pointage, demandes et notifications

Introduction

Ce sprint regroupe trois mécanismes transverses : le **pointage** de présence des employés, les **workflows de demandes** (création et modification) et le dispositif de **notifications** et de **journalisation** des actions.

6.1 Objectifs du Sprint

L'objectif est d'assurer le **suivi de présence** des employés, d'**encadrer** les opérations sensibles par des circuits de validation et de garantir la **traçabilité** des actions par la journalisation et les notifications.

6.2 Spécification des besoins

6.2.1 Diagramme de cas d'utilisation du Sprint 4

La figure 6.1 présente les cas d'utilisation du sprint.

6.2.2 Sprint Backlog du sprint 4

Le tableau 6.1 détaille les user stories et les tâches techniques.

Table 6.1 – Sprint Backlog du sprint 4

US	User story	Tâches techniques
US9	Pointer la présence (arrivée / départ).	Développer l'entité Pointage, le PointageService (EJB) et le PointageServlet (/pointage); déterminer le statut (PRESENT / RETARD au-delà de 9h); page pointage.jsp.
US10	Saisir ou corriger un pointage manuel.	Permettre à l'administrateur de saisir un pointage (PRESENT, RETARD, ABSENT) via PointageService.saveManuel.
US11	Soumettre une demande (client, employé, modification).	Développer les entités DemandeClient, DemandeEmploye, DemandeModification, DemandeAgence et les servlets DemandeAgenceServlet / MesDemandesServlet; statut EN_ATTENTE.
US12	Valider ou refuser une demande.	Développer le GestionModificationsServlet et le DemandeAgenceAdminServlet; faire évoluer le statut vers EFFECTUE / REFUSE; pages gestion-demandes.jsp, demandes-admin.jsp.
US13	Recevoir des notifications et consulter le journal d'audit.	Développer le NotificationService, l'entité Notification et le NotificationFilter; journaliser les actions via le JournalService (JournalAction).

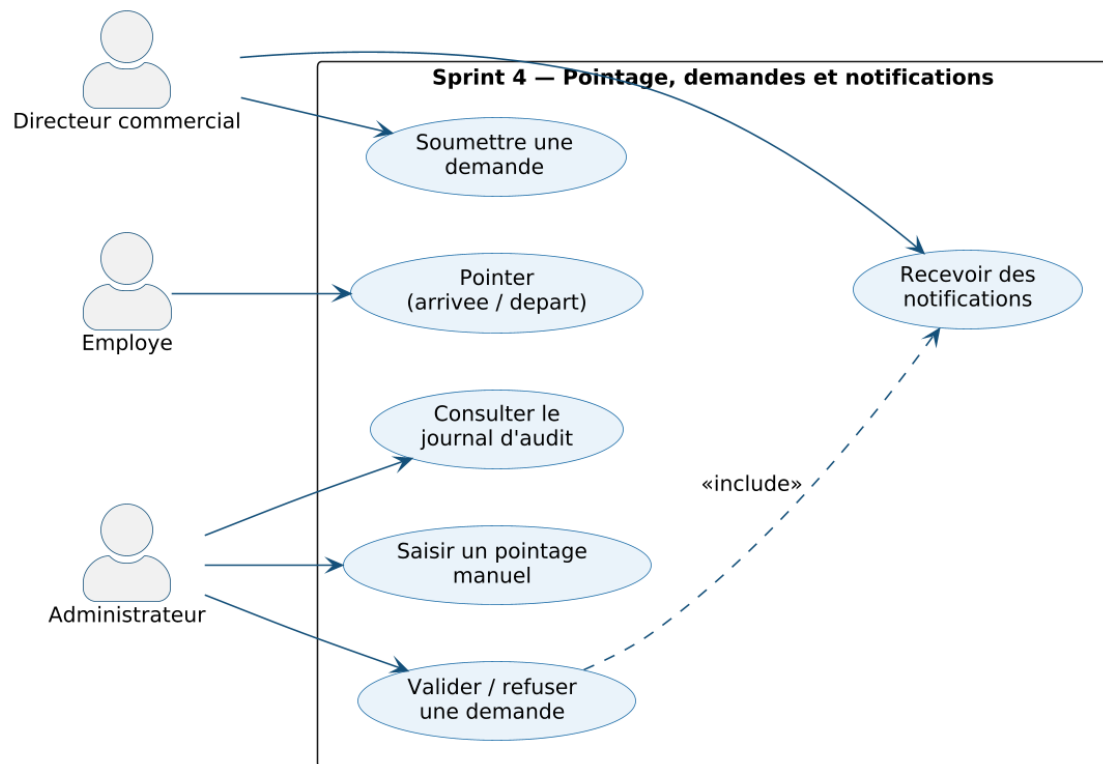


Figure 6.1 – Diagramme de cas d'utilisation du Sprint 4

6.3 Analyse des besoins

6.3.1 Description textuelle des cas d'utilisation

6.4 Conception

6.4.1 Diagramme de séquence : « Pointer un employé »

La figure 6.2 décrit le scénario de pointage.

Le `PointageServlet` transmet l'action au `PointageService`, qui recherche le pointage du jour. Le fragment `alt` distingue l'INSERT (premier pointage, statut PRESENT/RETARD au-delà de 9h) de l'UPDATE (pointage déjà existant).

6.4.2 Diagramme de séquence : « Valider une demande »

La figure 6.3 détaille le traitement d'une demande par l'administrateur.

Table 6.2 – Description textuelle du cas d'utilisation « Pointer la présence »

Titre	Pointer la présence
Objectif	Permettre à l'employé d'enregistrer son arrivée et son départ, avec détermination automatique du statut.
Acteur	Utilisateur (employé)
Pré-condition	L'employé doit être authentifié.
Post-condition	Le pointage du jour est enregistré avec son statut (PRESENT ou RETARD).
Scénario nominal	(1) L'employé clique sur « Pointer » ; (2) le PointageService relève l'heure courante et recherche le pointage du jour ; (3) s'il n'existe pas, il détermine le statut (RETARD au-delà de 9h) et l'enregistre ; (4) sinon, il met à jour l'enregistrement existant.

Table 6.3 – Description textuelle du cas d'utilisation « Valider une demande »

Titre	Valider une demande
Objectif	Permettre à l'administrateur d'approuver ou de refuser une demande soumise.
Acteur	Administrateur
Pré-condition	Une demande au statut EN_ATTENTE doit exister.
Post-condition	Le statut de la demande évolue vers EFFECTUE ou REFUSE et l'action est journalisée.
Scénario nominal	(1) L'administrateur consulte les demandes en attente ; (2) il sélectionne une demande ; (3) il l'approuve ou la refuse avec un commentaire ; (4) le service met à jour le statut, applique la modification le cas échéant et journalise l'action.

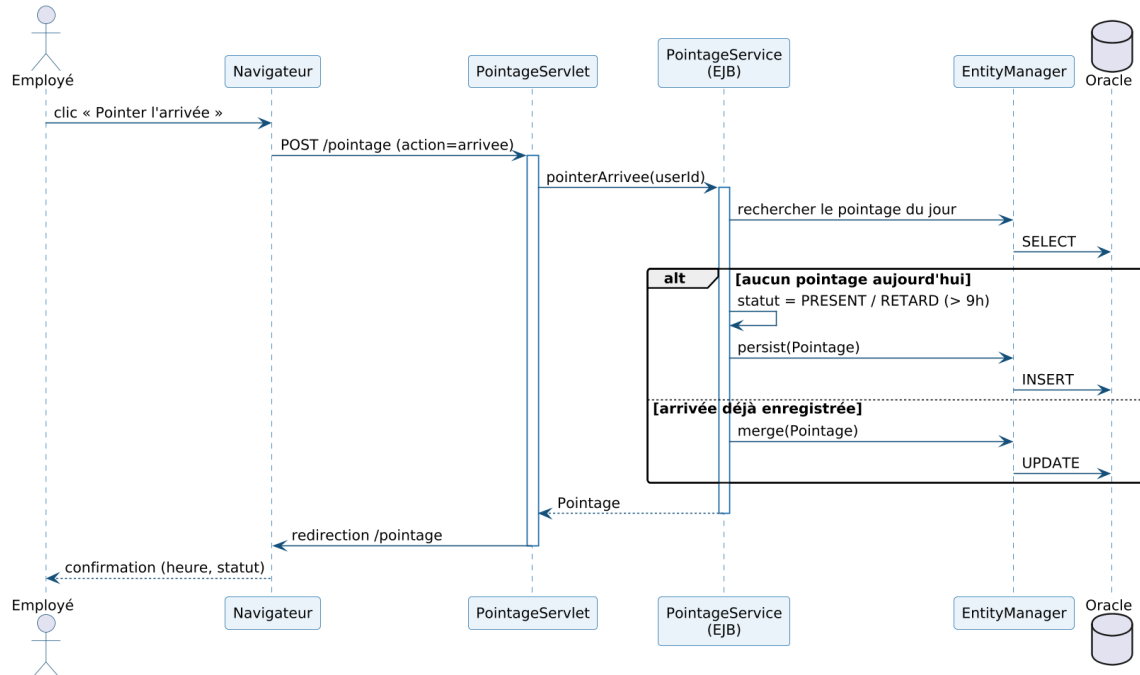


Figure 6.2 – Diagramme de séquence – Pointage d'un employé

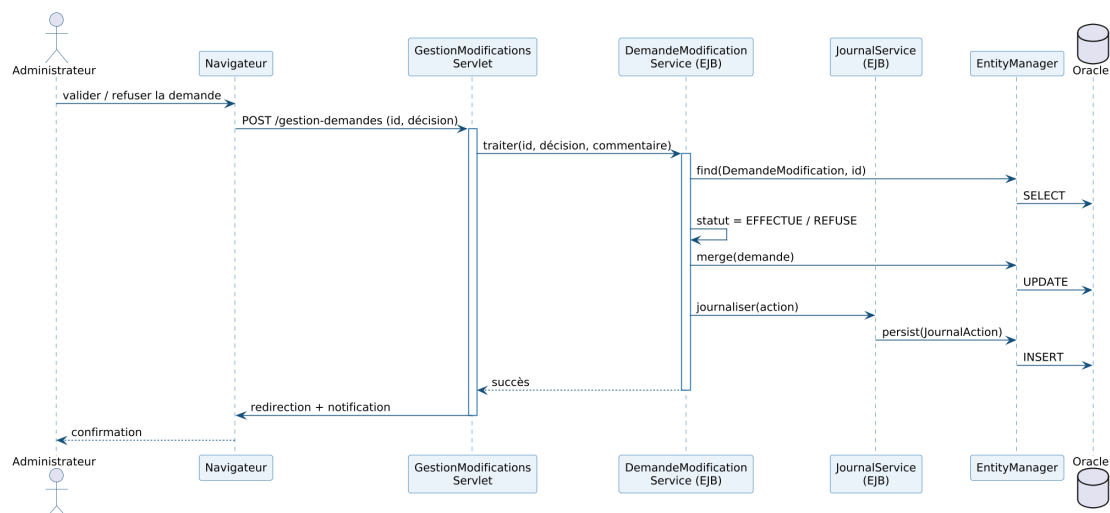


Figure 6.3 – Diagramme de séquence – Validation d'une demande

Le GestionModificationsServlet transmet la décision au service métier, qui charge la demande, met à jour son statut, persiste la modification puis journalise l'action via le JournalService.

6.4.3 Diagramme d'états-transitions : cycle de vie d'une demande

La figure 6.4 modélise le cycle de vie d'une demande.

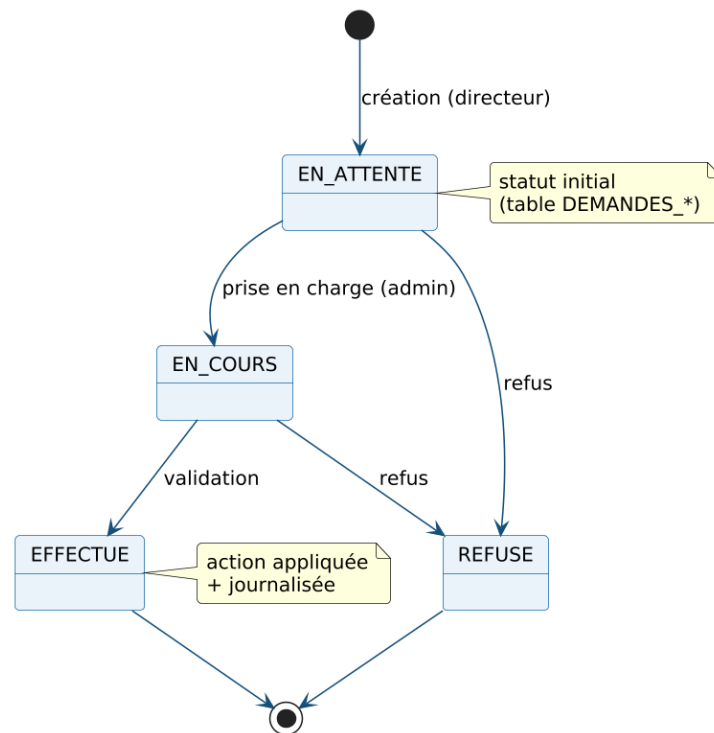


Figure 6.4 – Diagramme d'états-transitions – Cycle de vie d'une demande

Une demande naît à l'état EN_ATTENTE, passe en EN_COURS lors de sa prise en charge, puis atteint un état final EFFECTUE ou REFUSE, en cohérence avec la contrainte de la table des demandes.

6.4.4 Diagramme d'activité : traitement d'une demande

La figure 6.5 représente le processus métier de traitement d'une demande.

Le directeur soumet une demande, notifiée à l'administrateur. Le nœud de décision oriente le flux : en cas de validation, le statut passe à EFFECTUE et la modification est appliquée ; en cas de refus, le statut passe à REFUSE avec un commentaire. Les deux branches convergent vers la journalisation et la notification du demandeur.

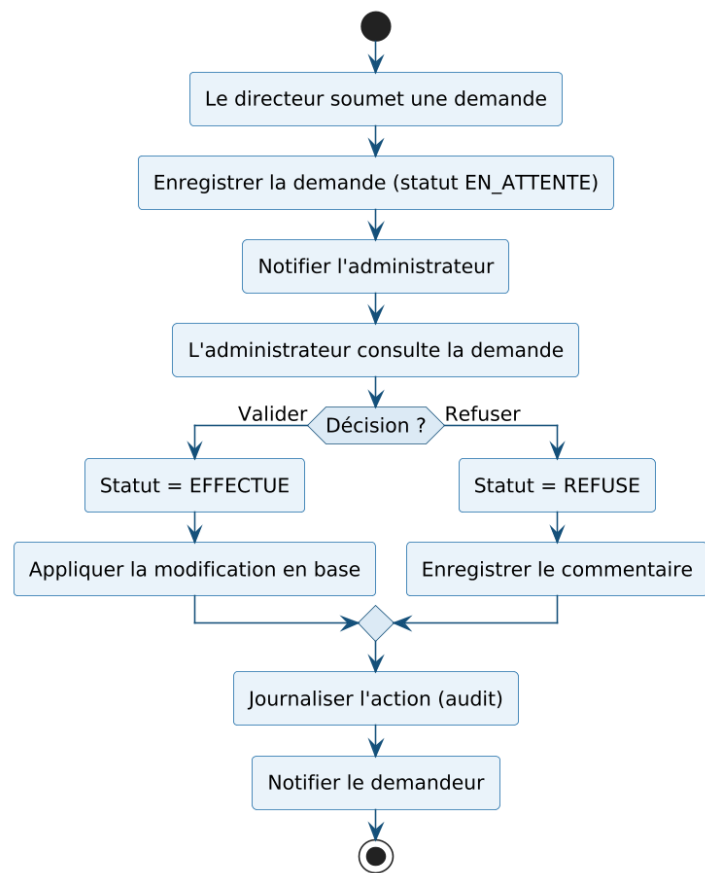


Figure 6.5 – Diagramme d'activité – Traitement d'une demande

6.5 Réalisation : interfaces utilisateur

La figure 6.6 présente l'interface du module de pointage : les cartes de synthèse (présents, retards, absents, taux de présence), le pointage du jour (boutons « Pointer l'arrivée / le départ ») et le formulaire de saisie ou de correction manuelle.



Figure 6.6 – Interface du module de pointage des employés

Conclusion

Ce sprint a doté l'application du suivi de présence, des circuits de validation et de la traçabilité des actions. Le sprint suivant met en place la chaîne ETL et le volet décisionnel.

Chapitre 7

Sprint 5 : Chaîne ETL et volet décisionnel

Introduction

Ce sprint réalise le **volet décisionnel** de la solution. Il met en place une chaîne **ETL** qui consolide les données opérationnelles dans un **datamart**, puis restitue les indicateurs au moyen d'un tableau de bord embarqué et de rapports Power BI.

7.1 Objectifs du Sprint

L'objectif est de **consolider** les données dispersées (employés, clients, objectifs, pointage) en un datamart agrégé par agence, et de **restituer** les indicateurs de pilotage aux responsables.

7.2 Spécification des besoins

7.2.1 Diagramme de cas d'utilisation du Sprint 5

La figure 7.1 présente les cas d'utilisation du sprint.

7.2.2 Sprint Backlog du sprint 5

Le tableau 7.1 détaille les user stories et les tâches techniques.

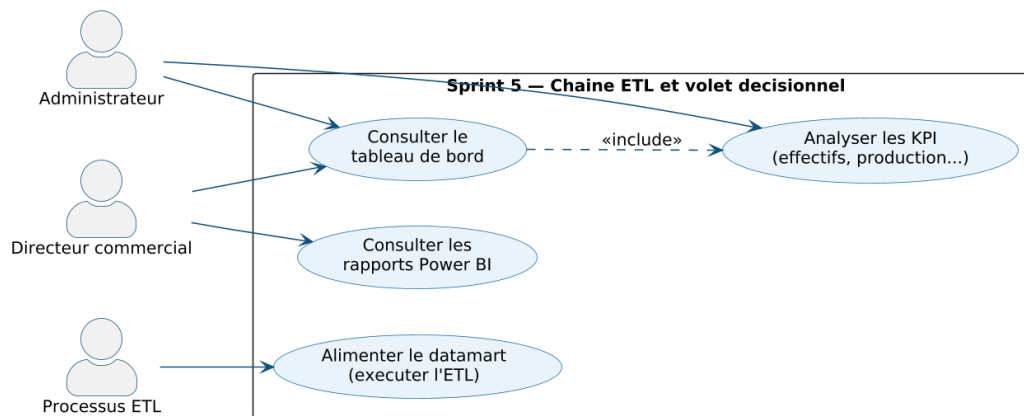


Figure 7.1 – Diagramme de cas d'utilisation du Sprint 5

Table 7.1 – Sprint Backlog du sprint 5

US	User story	Tâches techniques
US14	Alimenter le datamart via la chaîne ETL.	Développer le script Python <code>etl_agences.py</code> (pandas); étapes <i>Extract / Transform / Load</i> ; agréger les indicateurs par agence; produire le data-mart en étoile (<code>dim_agence</code> , <code>fait_agence</code>).
US15	Consulter le tableau de bord décisionnel.	Enrichir le <code>HomeServlet</code> avec les statistiques; réaliser le tableau de bord <code>home.jsp</code> (cartes d'indicateurs et graphiques ApexCharts).
US16	Consulter les rapports Power BI.	Créer les vues analytiques <code>V_BI_EMPLOYES</code> , <code>V_BI_CLIENTS</code> , <code>V_BI_OBJECTIFS</code> , <code>V_BI_POINTAGE</code> ; les connecter à Power BI.

7.3 Analyse des besoins

7.3.1 Description textuelle des cas d'utilisation

Table 7.2 – Description textuelle du cas d'utilisation « Alimenter le datamart (ETL) »

Titre	Alimenter le datamart (ETL)
Objectif	Consolider les données opérationnelles en un datamart agrégé par agence.
Acteur	Processus ETL / Administrateur
Pré-condition	Les données sources (agences, employés, clients, objectifs, pointage) doivent être disponibles.
Post-condition	Le datamart en étoile est produit et mis à la disposition de la restitution et de la segmentation.
Scénario nominal	(1) L'étape <i>Extract</i> lit les données sources ; (2) l'étape <i>Transform</i> les nettoie et les agrège par agence ; (3) l'étape <i>Load</i> écrit le datamart (dim_agence, fait_agence).

Table 7.3 – Description textuelle du cas d'utilisation « Consulter le tableau de bord »

Titre	Consulter le tableau de bord
Objectif	Restituer aux responsables une vision synthétique de l'activité.
Acteur	Administrateur / Directeur commercial
Pré-condition	L'acteur doit être authentifié.
Post-condition	Les indicateurs clés (effectifs, présence, production) sont affichés.
Scénario nominal	(1) L'acteur se connecte ; (2) le <code>HomeServlet</code> calcule les statistiques ; (3) la page <code>home.jsp</code> affiche les cartes d'indicateurs et les graphiques ApexCharts.

7.4 Conception

7.4.1 Chaîne décisionnelle et processus ETL

Les données nécessaires au pilotage sont dispersées dans les tables opérationnelles et exprimées à des grains hétérogènes. Le processus **ETL** (*Extract – Transform – Load*) les consolide en un datamart agrégé par agence. La figure 7.2 présente cette chaîne décisionnelle.

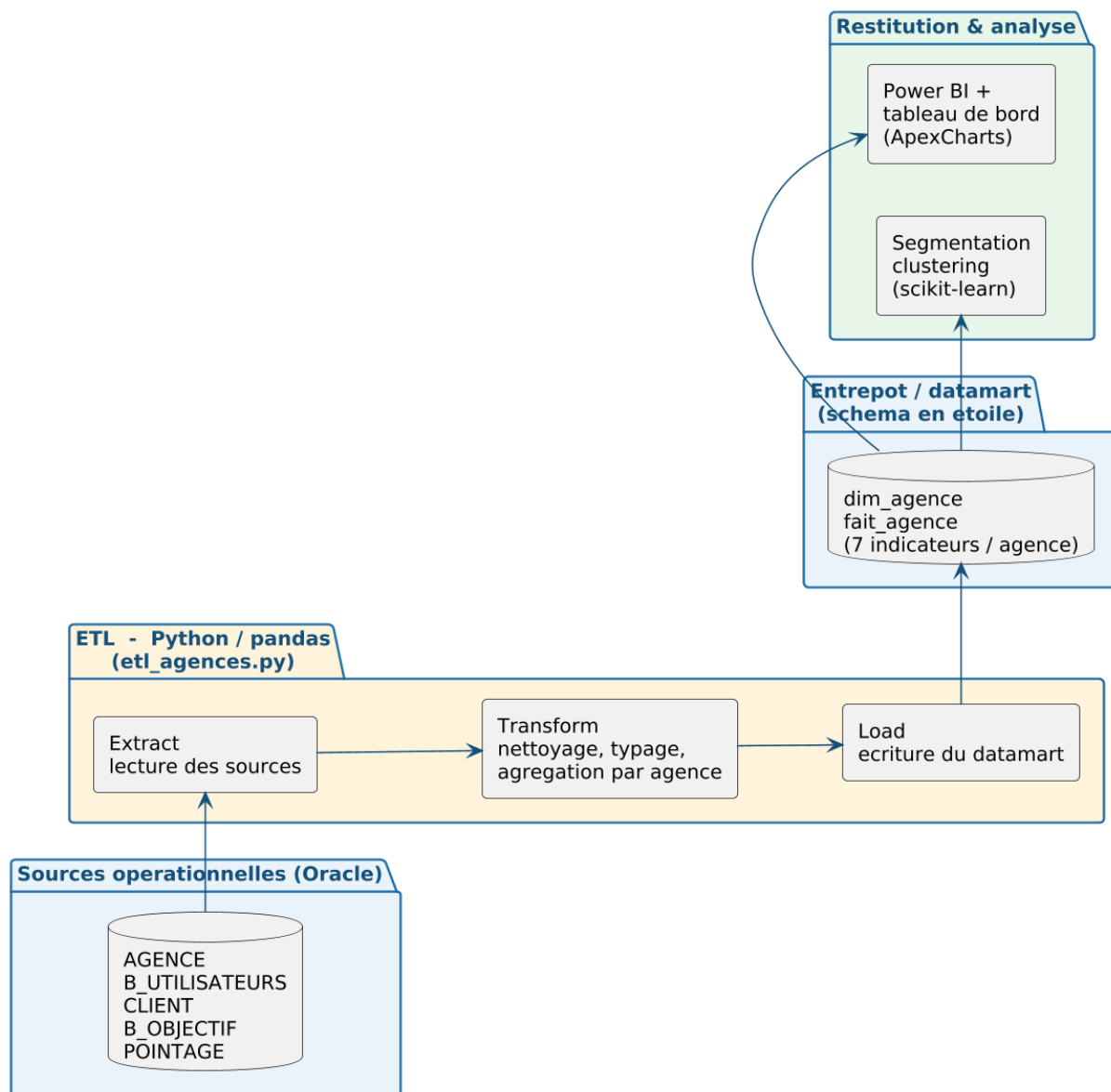


Figure 7.2 – Chaîne décisionnelle : du système opérationnel au datamart et à sa restitution

Trois étapes s'enchaînent : **Extract** (lecture des sources AGENCE, B_UTILISATEURS, CLIENT, B_OBJECTIF, POINTAGE), **Transform** (nettoyage, typage et agrégation par agence) et **Load** (chargement du datamart en étoile). Ce datamart constitue une **source unique consolidée**, partagée par la restitution (tableau de bord et Power BI) et par la segmentation des agences (chapitre 8).

7.4.2 Modèle en étoile de l'entrepôt

L'entrepôt est organisé selon une logique en **étoile** (figure 7.3) : autour de tables de faits (production, effectifs, clients, présence), des dimensions descriptives (agence, gestionnaire, période, type de client) permettent l'analyse multidimensionnelle. Les vues V_BI_* matérialisent cette couche sémantique pour l'outil de restitution.

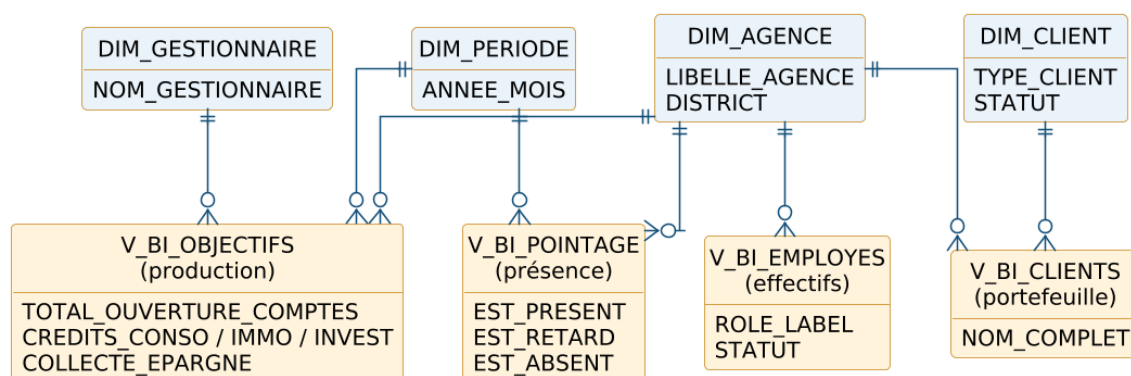


Figure 7.3 – Modèle décisionnel en étoile (vues V_BI_*)

Le tableau 7.4 récapitule les principaux indicateurs (KPI) retenus.

Table 7.4 – Indicateurs de performance (KPI) du volet décisionnel

Indicateur	Description	Source
Effectifs par agence	Nombre d'employés par agence	V_BI_EMPLOYES
Répartition des rôles	Gestionnaires / conseillers / hors commercial	V_BI_EMPLOYES
Portefeuille clients	Clients par type et par agence	V_BI_CLIENTS
Ouvertures de comptes	Comptes chèques + épargne + courants	V_BI_OBJECTIFS
Production de crédits	Conso, immobilier, investissement	V_BI_OBJECTIFS
Collecte d'épargne	Épargne additionnelle collectée	V_BI_OBJECTIFS
Taux de présence	Présents, retards et absences par agence et période	V_BI_POINTAGE

7.5 Réalisation

7.5.1 Mise en œuvre de la chaîne ETL en Python

La chaîne ETL est implémentée en **Python** (script `etl_agences.py`) à l'aide de la bibliothèque **pandas** [1]. Elle est organisée en trois fonctions : `extract()` lit les données sources (connexion Oracle via le connecteur `oracledb` ou, à défaut, export CSV); `transform()` nettoie et **agrège les données par agence** pour produire les sept indicateurs de pilotage; `load()` écrit le datamart en étoile (`dim_agence`, `fait_agence`) et le fichier réutilisé par la segmentation. L'extrait 7.1 illustre le cœur de l'agrégation.

Listing 7.1 – Agrégation des indicateurs par agence (étape Transform)

```
1  # effectif et nombre de questionnaires par agence
2  eff = employees.groupby("SK_AGENCE").agg(
3      effectif=("SK_UTILISATEUR", "count"),
4      nb_questionnaires=("EST_QUESTIONNAIRE", "sum")).reset_index()
5  # portefeuille clients
6  cli = clients.groupby("SK_AGENCE").size().reset_index(name="nb_clients")
7  # production (comptes, crédits, épargne)
8  obj = objectifs.groupby("SK_AGENCE").agg(
9      total_comptes=("COMPTES", "sum"),
10     production_credits=("CREDITS", "sum"),
11     collecte_epargne=("EPARGNE", "sum")).reset_index()
12 # taux de présence issu du pointage
13 poi = pointages.assign(
14     present=pointages["STATUT"].isin(["PRESENT", "RETARD"]).astype(int) \
15     .groupby("SK_AGENCE").agg(taux_presence=("present", "mean")).reset_index()
```

L'exécution du script produit le datamart sous `etl/entrepot/` (`dim_agence.csv`, `fait_agence.csv`) ainsi que le fichier consolidé `clustering/data/agences.csv`. Le pipeline est **rejouable** et journalise le nombre d'enregistrements traités à chaque étape.

7.5.2 Tableau de bord et restitution Power BI

Le tableau de bord embarqué (`home.jsp`, alimenté par le `HomeServlet`) comporte une rangée de **cartes d'indicateurs** (employés, clients, présents du jour, taux de présence, de-

mandes en attente) puis des **graphiques** ApexCharts (effectif par agence, répartition des rôles, présence du jour). En complément, les vues V_BI_* sont connectées à **Power BI** pour des rapports interactifs. La figure 7.4 présente le tableau de bord embarqué.

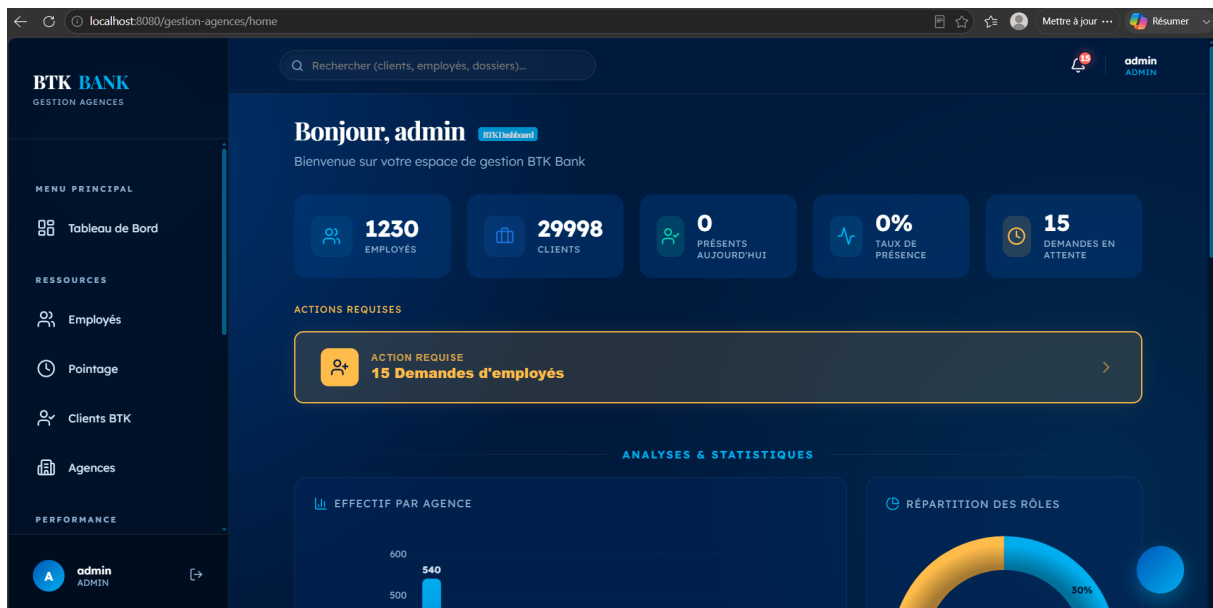


Figure 7.4 – Tableau de bord décisionnel embarqué de l'application

Conclusion

Ce sprint a doté la solution d'un volet décisionnel complet : une chaîne ETL alimente un datamart consolidé, restitué par un tableau de bord et par Power BI. Le datamart ainsi produit constitue l'entrée du dernier sprint, consacré à la segmentation des agences par apprentissage automatique.

Chapitre 8

Sprint 6 : Segmentation des agences par apprentissage non supervisé

Introduction

Ce dernier sprint introduit une brique d'**intelligence artificielle** : la segmentation des agences par **apprentissage non supervisé** (*clustering*). À partir du datamart produit par la chaîne ETL, il regroupe automatiquement les agences en profils homogènes afin d'éclairer les décisions de pilotage.

8.1 Objectifs du Sprint

L'objectif est de **segmenter les agences** présentant des caractéristiques d'activité similaires, sans étiquette préalable, afin d'appuyer le ciblage, l'allocation des ressources et la définition d'objectifs différenciés.

8.2 Spécification des besoins

8.2.1 Diagramme de cas d'utilisation du Sprint 6

La figure 8.1 présente les cas d'utilisation du sprint.

8.2.2 Sprint Backlog du sprint 6

Le tableau 8.1 détaille la user story et les tâches techniques.

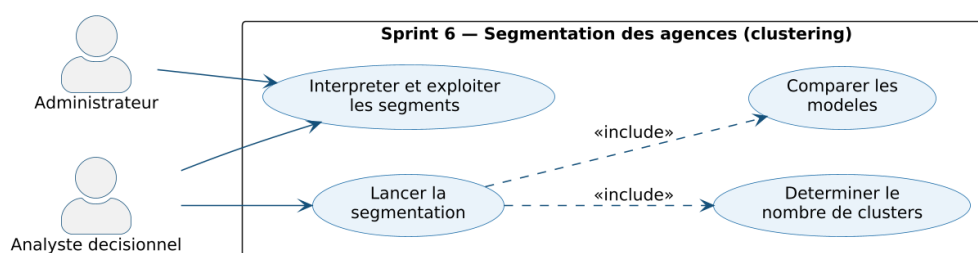


Figure 8.1 – Diagramme de cas d'utilisation du Sprint 6

Table 8.1 – Sprint Backlog du sprint 6

US	User story	Tâches techniques
US17	Segmenter les agences par apprentissage non supervisé.	Développer le script <code>segmentation_agences.py</code> (scikit-learn); charger le datamart et standardiser les variables; déterminer le nombre de clusters (coude + silhouette); comparer plusieurs modèles; projeter en PCA et profiler les segments.

8.3 Analyse des besoins

8.3.1 Description textuelle du cas d'utilisation

Table 8.2 – Description textuelle du cas d'utilisation « Segmenter les agences »

Titre	Segmenter les agences
Objectif	Regrouper automatiquement les agences en profils homogènes à partir de leurs indicateurs d'activité.
Acteur	Analyste décisionnel / Administrateur
Pré-condition	Le datamart agrégé par agence (<code>fait_agence</code>) doit être disponible.
Post-condition	Les agences sont réparties en segments cohérents, caractérisés et exploitables pour la décision.
Scénario nominal	(1) L'analyste lance la segmentation; (2) le script standardise les variables; (3) il détermine le nombre de clusters puis compare les modèles; (4) il retient le meilleur modèle, projette en PCA et restitue le profil de chaque segment.

8.4 Conception : démarche de clustering

8.4.1 Données et variables

Les données ne sont pas extraites directement de la base opérationnelle : elles proviennent du **datamart** produit par la chaîne ETL (table de faits *fait_agence*). Chaque agence y est décrite par sept indicateurs agrégés (tableau 8.3), ce qui garantit la cohérence entre le tableau de bord, Power BI et l'analyse non supervisée.

Table 8.3 – Variables de segmentation (par agence)

Variable	Description
effectif	Nombre d'employés de l'agence
nb_gestionnaires	Nombre de gestionnaires
nb_clients	Taille du portefeuille clients
total_comptes	Total des ouvertures de comptes
production_credits	Production de crédits
collecte_epargne	Collecte d'épargne
taux_presence	Taux de présence (issu du pointage)

8.4.2 Prétraitement

Les variables étant exprimées dans des ordres de grandeur très différents (un effectif de quelques dizaines face à des milliers de comptes), une **standardisation** (centrage-réduction, *StandardScaler*) est appliquée. Sans elle, les variables de forte amplitude domineraient le calcul des distances et fausseraient le regroupement.

8.4.3 Détermination du nombre de clusters

Le nombre de clusters k étant inconnu a priori, il est déterminé en combinant la **méthode du coude** (inertie intra-cluster) et le **score de silhouette** (figure 8.2).

Les deux indicateurs concordent : l'inertie présente un **coude marqué** à $k = 3$ et le score de silhouette y atteint son **maximum** ($\approx 0,74$). La segmentation en **trois clusters** est donc retenue.

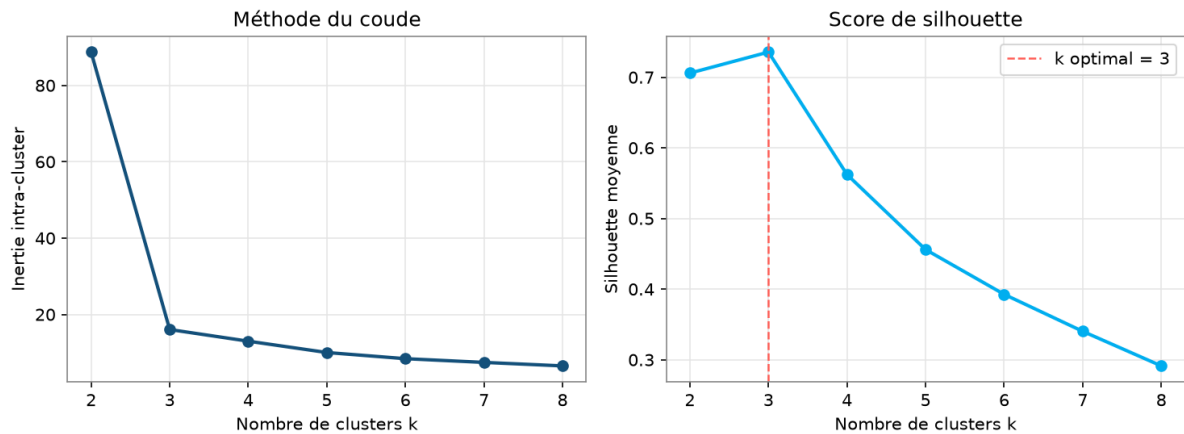


Figure 8.2 – Choix du nombre de clusters : méthode du coude et score de silhouette

8.4.4 Comparaison des modèles de clustering

Quatre algorithmes ont été comparés : **K-Means**, **classification ascendante hiérarchique** (agglomératif), **modèle de mélange gaussien** (GMM) et **DBSCAN**, à l'aide de trois indices internes (tableau 8.4, figure 8.3).

Table 8.4 – Comparaison des modèles de clustering

Modèle	Nb clusters	Silhouette ↑	Davies-Bouldin ↓	Calinski-Harabasz ↑
K-Means	3	0,736	0,350	391
Agglomératif	3	0,736	0,350	391
GMM	3	0,736	0,350	391
DBSCAN	3	0,736	0,350	391

Les quatre algorithmes **convergent vers la même segmentation** : ils identifient les trois mêmes clusters et obtiennent des indices identiques (silhouette 0,736, Davies-Bouldin 0,350, Calinski-Harabasz 391). Cette concordance, y compris pour des approches de nature différente (partitionnement, hiérarchique, probabiliste et à base de densité), confirme la **robustesse** des trois segments. Le **K-Means** est retenu : il atteint le meilleur score tout en restant simple et interprétable.

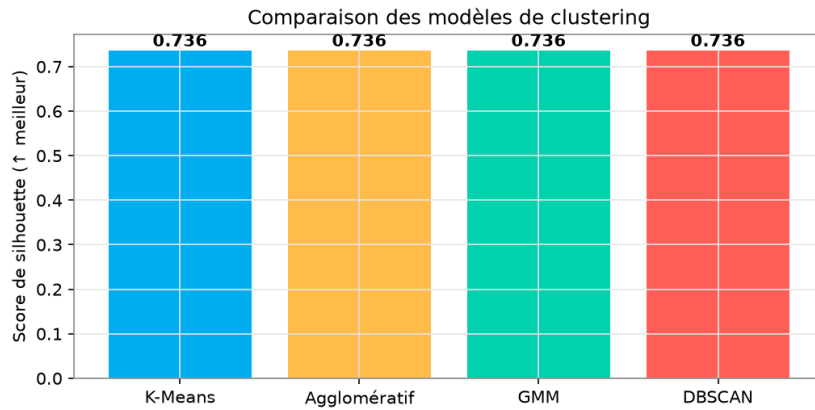


Figure 8.3 – Comparaison des modèles selon le score de silhouette

8.5 Réalisation : résultats et interprétation

La figure 8.4 projette les agences dans le plan des deux premières composantes principales (PCA), colorées selon leur cluster ; la première composante, qui résume la *taille* de l'agence, explique l'essentiel de la variance. Les trois segments sont nettement séparés.

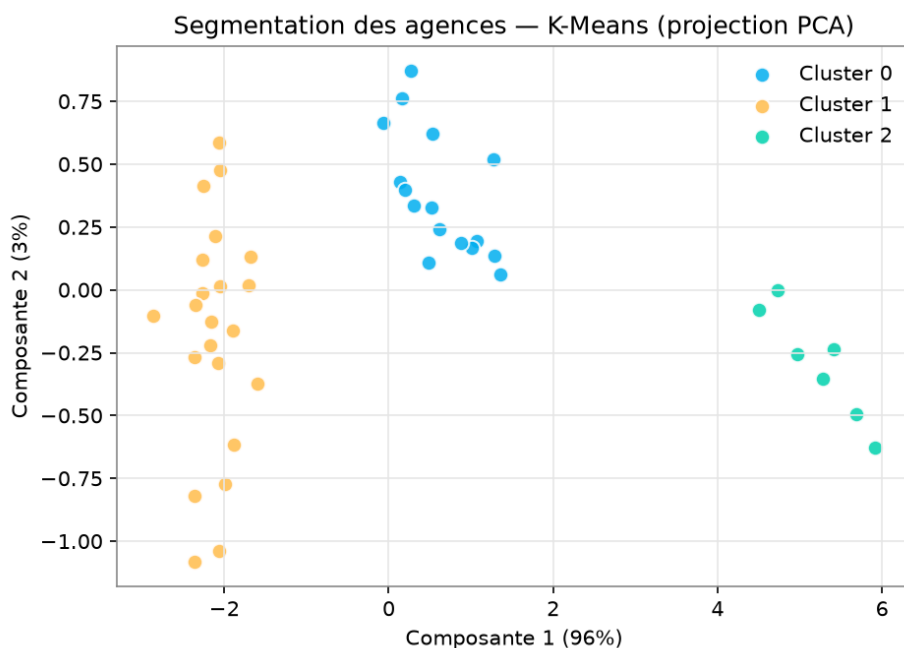


Figure 8.4 – Segmentation des agences (K-Means) projetée par PCA

Le profil moyen de chaque segment (tableau 8.5) permet de les caractériser.

On distingue ainsi un segment de **grandes agences** (pôles majeurs à fort effectif et forte production), un segment d'**agences moyennes** (activité intermédiaire) et un segment de **pe-**

Table 8.5 – Profil moyen des segments d'agences

Segment	Nb	Effectif	Clients	Comptes	Prod. crédits	Présence
Grandes agences	7	116	908	2333	1734	0,98
Moyennes agences	16	54	438	1076	823	0,94
Petites agences	22	21	160	431	321	0,87

tités agences (effectif et activité réduits, taux de présence plus faible).

8.5.1 Apport décisionnel

Cette segmentation constitue un outil d'**aide à la décision** : elle permet d'adapter les actions à chaque profil — allouer les ressources selon la taille, fixer des objectifs différenciés, accompagner spécifiquement les petites agences ou capitaliser sur les grandes. Elle prolonge le volet décisionnel et illustre l'apport de l'**intelligence artificielle** au pilotage du réseau.

Conclusion

Ce sprint a enrichi la solution d'une dimension analytique avancée. Après détermination du nombre optimal de clusters et comparaison de plusieurs modèles, le K-Means a fait émerger trois segments d'agences cohérents et exploitables pour la décision.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et de réaliser, au sein de la **BTK – Banque Tuniso-Koweïtienne**, une application web de gestion des agences bancaires adossée à un volet décisionnel. Conduit selon la méthode **Scrum**, le travail a été découpé en sprints livrant chacun un incrément fonctionnel : l'authentification et la gestion des rôles, la gestion des agences et des employés, celle des clients et des objectifs, le pointage et les workflows de demandes, la chaîne ETL et le volet décisionnel, et enfin la segmentation des agences.

La solution centralise la gestion des agences, des employés, des clients et des objectifs, formalise les opérations sensibles au moyen de circuits de validation, assure le suivi de présence des employés et restitue aux responsables des indicateurs d'aide à la décision via un tableau de bord embarqué et des rapports Power BI. Elle intègre en outre une chaîne **ETL** qui consolide les données dans un datamart, ainsi qu'une brique d'**intelligence artificielle** : la segmentation des agences par apprentissage non supervisé (clustering).

Sur le plan technique, ce travail a été l'occasion de mettre en pratique les compétences acquises durant la formation : modélisation UML, développement Jakarta EE (Servlets, JSP, EJB, JPA), conception de bases de données Oracle, mise en œuvre d'une chaîne décisionnelle (ETL en Python, vues analytiques et *reporting* Power BI), ainsi qu'une initiation à l'**apprentissage automatique**, au croisement du développement logiciel et de la Business Intelligence.

Plusieurs perspectives d'évolution peuvent être envisagées :

- le chiffrement (hachage) des mots de passe et le renforcement de la sécurité ;
- l'**industrialisation** de la chaîne ETL : ordonnancement automatique et historisation incrémentale des indicateurs ;
- l'enrichissement par des analyses prédictives (prévision des objectifs) ;
- l'exposition d'une API REST pour l'interopérabilité avec d'autres systèmes ;
- le développement d'une application mobile de consultation des indicateurs.

Nétographie

- [1] The pandas development team, "pandas – python data analysis library," 2025. <https://pandas.pydata.org/docs/>.
- [2] Eclipse Foundation, "Jakarta ee 10 – documentation officielle," 2025. <https://jakarta.ee/>.
- [3] Red Hat, "Hibernate orm – documentation," 2025. <https://hibernate.org/orm/documentation/>.
- [4] Oracle Corporation, "Oracle database – documentation," 2025. <https://docs.oracle.com/en/database/>.
- [5] Red Hat, "Wildfly application server – documentation," 2025. <https://docs.wildfly.org/>.
- [6] Microsoft, "Microsoft power bi – documentation," 2025. <https://learn.microsoft.com/power-bi/>.
- [7] ApexCharts, "Apexcharts.js – documentation," 2025. <https://apexcharts.com/docs/>.
- [8] Apache Software Foundation, "Apache maven – documentation," 2025. <https://maven.apache.org/guides/>.
- [9] BTK Bank, "Banque tuniso-koweïtienne – site officiel," 2025. <https://www.btknet.com/>.
- [10] scikit-learn developers, "scikit-learn – machine learning in python," 2025. <https://scikit-learn.org/stable/>.

Annexe A

Annexes

A.1 Scripts SQL d'initialisation

Les scripts de création des tables et des séquences (sécurité, demandes, modifications) ainsi que les définitions des vues décisionnelles (V_BI_EMPLOYES, V_BI_CLIENTS, V_BI_OBJECTIFS) figurent dans le dépôt du projet, sous le répertoire sql/.

A.2 Captures complémentaires

Des captures d'écran complémentaires de l'application et des rapports Power BI pourront être insérées dans cette annexe.

A.3 Dépôt du code source

Le code source complet du projet est disponible sur le dépôt GitHub associé au rapport.